MASTER

Combustion (Gaseous) of Pulverised Coal Particles in the Injection Zone of a Blast Furnace Using FGM on OpenFOAM

Pashikanti, R.

Award date:
2021

Department of Mechanical Engineering

Master's Project

Combustion (Gaseous) of Pulverised Coal Particles in the Injection Zone of a Blast Furnace Using FGM on OpenFOAM

Student	-	Rohan Pashikanti	Master	-	Mechanical Engineering
Identity No.	-	1357352	Department	-	Mechanical Engineering
Supervisor 1	-	Dr. Yali Tang	Research Group	-	Power and Flow
Supervisor 2	-	ir. Chih-Chia Huang	Date	-	January 29, 2021

Technische Universiteit
Eindhoven
University of Technology

Declaration concerning the TU/e Code of Scientific Conduct for the Master's thesis

I have read the TU/e Code of Scientific Conduct[i].

I hereby declare that my Master's thesis has been carried out in accordance with the rules of the TU/e Code of Scientific Conduct

Date

20 - January - 2021

Name

Rohan Pashikanti

ID-number

1357352

Signature

Submit the signed declaration to the student administration of your department.

[i] See: http://www.tue.nl/en/university/about-the-university/integrity/scientific-integrity/
The Netherlands Code of Conduct for Academic Practice of the VSNU can be found here also.
More information about scientific integrity is published on the websites of TU/e and VSNU

Abstract

In order to reduce coke consumption in modern blast furnaces, the technology of injecting pulverised coal particles has proven to be an effective improvement in the recent decades, enriching the iron making process. The coal combustion inside the raceway generates heat and reducing gases which melt and oxidise the ore to produce iron. Lack of actual real-time measurement techniques due to harsh conditions of the raceway make it difficult to investigate the material conversion in raceway to minimize dust generation and improve efficiency of iron-making process. Numerical methods provide a cost effective way to simulate and investigate raceway phenomena.

This study aims to simulate volatile matter combustion released from coal. A chemical reduction technique by the name of FGM (Flamelet Generated Manifold) is adapted to bring down the computational cost of the raceway simulation. A rudimentary adaption of FGM on OpenFOAM is realised. In FGM (Flamelet Generated Manifold) method, a 3D flame is considered to be an ensemble of 1-D flames. To realise this method, a manifold is assembled with a few controlling variables and all the other thermo-chemical properties are stored as a function of these controlling variables. During flame simulations the database is looked up instead of solving for all the thermo-chemical variables. The generation and implementation of the FGM method is shown in this study and finally a future projection is made to realise this study's potential for incorporation in a full comprehensive raceway model.

Acknowledgements

I would first like to thank my daily supervisor ir. Chih-Chia Huang from the Power and Flow research group at TU Eindhoven. He has been a source of inspiration in the way he conducted work and tackled problems. Thank for being just a message away for anything related to the project or any topic in general. A special thanks to my mentor and my supervisor Dr. Yali Tang. She has been a constant motivator throughout my master's course. The door to Dr. Tang's office was always open whenever I had run into any sort of difficulty. I was blessed to have an amazing pair of proactive and optimistic supervisors who always believed in me and gave me confidence throughout.

I greatly acknowledge the vital assistance of the members from the CoCo team : Prof. Jeroen van Oijen, ir. Nithin Mukundakumar, ir. Thijs Hazenberg, M.Sc. Abhijit Kalbhor, ir. Hesheng Bao and M.Sc. Suleyman Karaca from the Power and Flow research group for always helping me out with my project whenever I sought help.

I extend my thanks to the graduation committee : Prof. Jeroen van Oijen, Dr. Yali Tang, Dr. Arjan Frijns and advisor the the committee, ir. Chih-Chia Huang for taking time and going through the evaluation process of the project.

I am fortunate and grateful to have found my student family in Eindhoven: Pranav, Piyushi, Craig, Anirudh and Shrivaas whom I love sharing trips, food and stories with, my closest people halfway across the planet : Payal, Akhil, Samarth, Hitesh, Pooja, Tanya, Vaman, Jacob, Kiran for always cheering for me and lifting me up throughout my period of study.

Finally, I must express my profound gratitude to my parents, brother and my cousins for providing me with unfailing support and continuous encouragement through the tough covid times. My project would have not been possible without their sacrifice and backing.

Thank You.

Rohan Pashikanti

Preface

This report has been written under the M.Sc. Mechanical Engineering Programme, final semester in the Eindhoven University of Technology, The Netherlands.

This report consists of two parts: the Main report and an Appendix.

Tables and figures have been enumerated with a number in chronological order of appearance and a list of figures/tables is added to the report.

Citations in the report have been made according to the ACS style.

Nomenclature

$\dot{\omega}$	Source term [kg/m^3$\cdot s$]
η	Dynamic Viscosity [kg$\cdot m^{-1} \cdot s^{-1}$]
λ	Thermal conductivity [W/m^2$\cdot K$]
Φ, CV	Control Variable [-]
ρ	Density [kg/m^3]
σ	Area [m^2]
$\vec{\tau}$	Stress tensor [N/m^2]
$\vec{g}$	Gravitational acceleration constant [m/s^2]
$\vec{q}$	Heat flux [W/m^2]
$\vec{U_i}$	Diffusion velocity of species i [m/s]
$\vec{u}$	Bulk velocity [m s^{-1}]
A, β	Pre-exponent factor and exponent factor [-]
C_p	Heat capacity [J/K]
D_i	Diffusion coefficient (mass) [-]
E_a	Activation energy [J/mol]
h	Specific enthalpy [kJ/kG]
k_f, k_r	reaction rate coefficient; forward, reverse [L$\cdot mol^{-1} \cdot s^{-1}$]
Le_i	Lewis number of species i
N_s, N_e	Number of species, Number of elements [-]
R	Universal gas constant [J K^{-1}$\cdot mol^{-1}$]
r_f, r_r	Reaction rate; forward, reverse [mol L^{-1} s^{-1}]

v	Stoichiometric coefficient [-]
Y_i	Mass fraction of species i [-]
Z	Mixture Fraction [-]
p	Pressure [N/m^2]
t	Time [s]

List of Figures

List of Tables

Contents

Chapter 1

Introduction

With a constant figure, close to 150 million tonnes of steel is produced every year just in the European Union alone [1], out of which 70% is produced by the convectional blast furnace or the blast furnace - oxygen steel making route. The other route being electric-arc steel making which accounts for less than 30 % of the total steel produced [2]. The convectional blast furnace steel making procedure is explained in brief in section 1.1. Coke and coal are the main fuel sources and the reducing agents, while ore and sinter are the major steel sources for the oxygen-steel-making process. As coal and coke reduce in the furnace, they are oxidised to mainly CO. This CO reduces the iron ore and and CO_2 is released. Giving importance to the ever-rising CO_2 levels in the environment, constant efforts are being made to improve and optimize the blast furnace to make the process much more efficient and clean. This study is laid out with an effort to aid in the process of lowering the CO2 levels released by the blast furnace in the future.

1.1 Blast Furnace

The blast furnace is a counter flow chemical reactor in which iron oxides are reduced to liquid iron. It is a huge cylindrical well-type structure lined with refractory bricks in which different raw materials are charged from the top and liquid metal is drawn at the bottom. At the top of the furnace, iron ore (iron oxides) and coke are charged in layers. These layers together are called as the ‘burden’. There are a set number of equally-spaced small openings at the bottom of the blast furnace wall known as ‘Tuyeres’. Air and pulverised coal particles are injected into the furnace through the tuyere at a very high temperature (approx. 1200°C). Figure 1.1 shows the cross-sectional schematic of a typical blast furnace. The different layers on the top show the coke and the iron ore layers charged from the top. The coal particles combust near the tuyere where the reducing gases are formed from the coal in the form of CO.

The combustion zone just inside the furnace inlet from the tuyere forms a balloon like structure called a raceway where injected coal burns and coke is converted to CO and CO_2. These reducing gases travel upwards towards the top, reducing the ore near the cohesive zone which consists of the dripping zone and the softening zone. Liquid metal from the cohesive zone drips down and the slag separates from the liquid metal at the bottom-most part of the furnace called the "Hearth". This liquid metal is tapped out from the bottom and is sent to processing in a seperate facility. There are a huge number of chemical and physical processes simultaneously occurring in the blast furnace at any instant in time and the sheer size of the furnace and the extreme conditions in terms of flow and temperature make it very hard till date to interpret the exact inside of the blast furnace. And since the inside of the furnace cannot be "seen", as many experimental probes are not built for the extreme conditions inside the raceway. Hence, many questions still remain about the effect of raceway formation and combustion on the efficiency of the blast furnace.

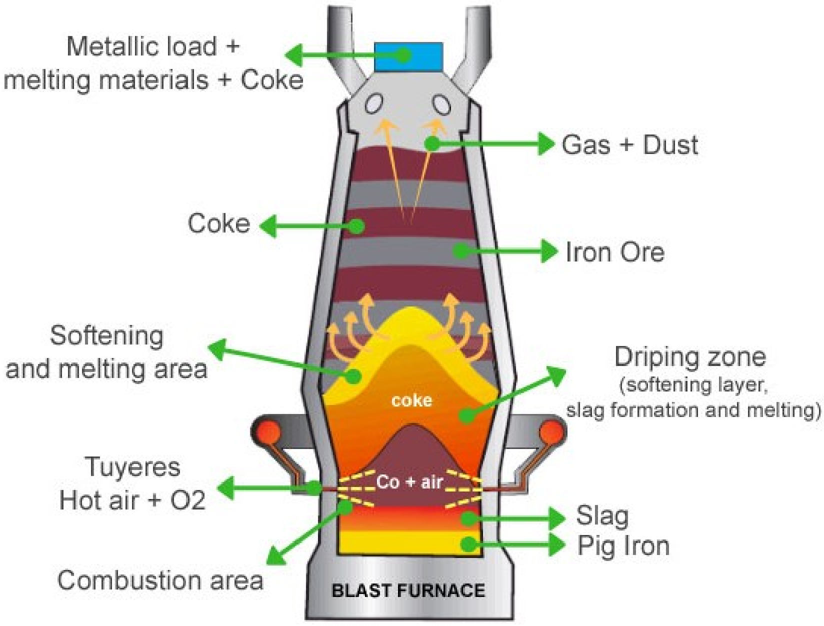


Figure 1.1: Blast Furnace Schematic [2]

1.1.1 Injection Zone

The injection zone consists of the tuyere and the raceway which is shown in figure 1.2. Fuel in the form of pulverised coal is injected through the tuyeres along with a stream of fast moving hot gas or "hot blast". Coal in this hot blast devolatises and produces volatile gases which combust and form mainly CO in front of the tuyere. The combustion of these volatile gases generate a hot flame creating a "raceway". A raceway can be explained as a void in the blast furnace where a bulk of pulverised coal and coke undergo phase change including devolatisation and gasification. This is also the region of highest temperature due to the combustion of volatiles and gasification of char. The heat required to melt the ore is generated in the reaceway. The hot reducing gases which are produced in the raceway mainly CO and CO_2 quickly travel upwards through the different layers of coke and ore, heating and reducing them as they travel to produce CO2 [2].

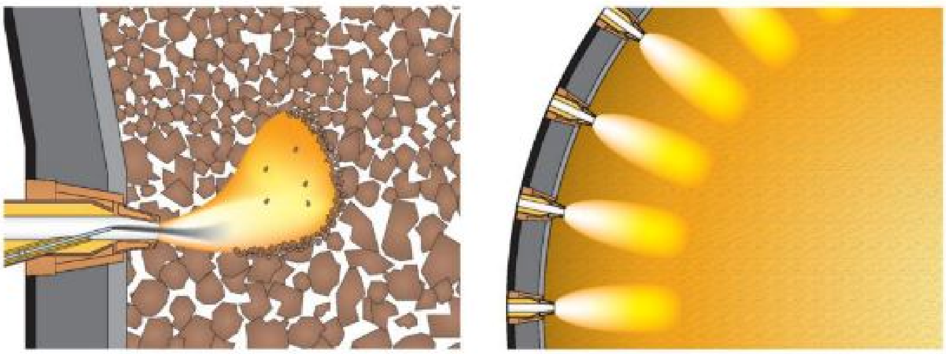


Figure 1.2: Horizontal and vertical sections of raceway formation [2]

The formation of this raceway depends on different operating parameters. The prediction of the exact shape and size of this raceway is a very complex task owing to severe conditions in a real furnace. There are studies that investigate the formation of raceway [3–5] and the work by Cui et. al. (2019) [6] captures the effects of the different operating parameters collectively using a CFD-DEM model of a furnace.
The different operating parameters that are taken into consideration are the inlet blast velocity, discharge rate of coal and the packed bed height. It is seen that when the inlet velocity of the blast is increased until it reaches the critical velocity that can fluidize the bed, the raceway size increases with an increasing inlet blast velocity. Similar trend is noticed for discharge rate, where an increase in discharge rate increases the raceway size due to higher number of particles breaking the interaction forces in the coke bed. As discharge rate increases, the amount of pulverised coal increases and so does the amount of volatile matter which ignites and provides has more energy to rearrange the coke bed and the gas flow. The size of the raceway also depends on the packed bed height. As the bed height increases the resistance offered to the gas flow gets higher, as a result the size of the raceway gets smaller. This happens until a critical bed height is reached after which the size of the raceway for an inlet velocity ceases to change [7]. The active coke region where the ore melts (also called the melting zone) is directly proportional to the size of the raceway [8]. This region impacts the production of iron in a furnace, since a larger active zone meaning higher output of molten metal. A larger and a stable melting zone is more productive than a smaller active region or an unstable raceway. Hence a stable raceway formation is of importance which does not collapse at low injection speeds due to the high bed pressure and which does not extend too far from the tuyere. A raceway that does not extend far enough will reduce the active region of coke which decreases the efficiency of the blast furnace.

1.1.2 Pulverised Coal Injection

For a blast furnace in the past which only used coke as fuel, the coke rate was typically in the range of 500-520 kg/tonnes-HotMetal. Modern blast furnaces use supplementary injectants via the tuyeres such as natural gas, oil and/or coal. This significantly reduces the coke rate to below 330 kg/tonnes-HotMetal if only coal is injected. Since a major part of hot metal cost depends on coke, the use of these injection materials are economically very feasible. This led to development of pulverised coal injection in the modern blast furnace [2].

Coal is injected via a lance into the tuyere, which gasifies and ignites in the raceway. The residence time of coal is very short in the raceway (approx. 5-10 ms). This gasification occurs as various steps. When compared to the other processes like the top to down flow of an ore/coke layer (approx. 8-10 hours) [9] or the travel of reducing gases (approx. 13 s) [10] from bottom to top of the furnace, it is very fast and requires its own comprehensive model which have been studied extensively and provided insight about the mechanics of coal gasification. First the coal when injected with the hot blast loses any moisture present, after which the release of volatile matter occurs also known as devolatization. The gaseous volatile matters ignite which causes a very sharp increase in the temperature in quick time. These steps are depicted in figure 1.3. These combustion reactions are very complex due to the different type of participating elements present which are not strictly uniform, such as ore, sinter, pellets, etc. A detailed model describing pulverised coal devolatisation and gasification [11] is needed to depict the processes in detail.

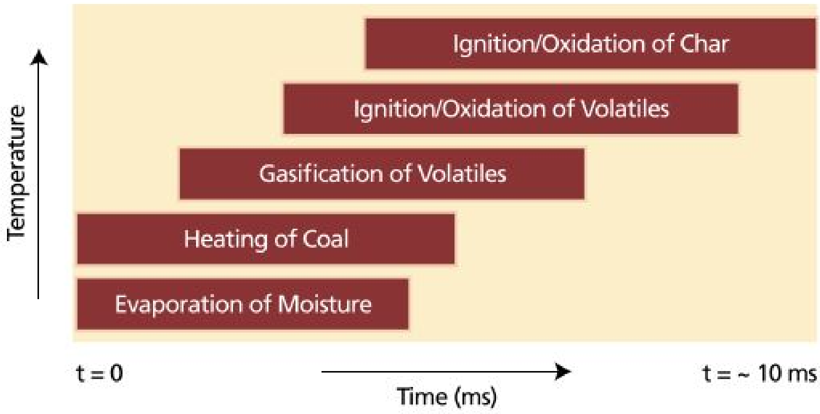


Figure 1.3: Coal gasification schematic [2]

1.1.3 Numerical Methods

The combustion process of coal is a very complex one involving a large number of reactions and interactions with different metal oxides in the furnace. Although experiments help us study these processes in detail singularly and generally very expensive, to model these collectively and to understand the mechanics of coal combustion and the role it plays in metal making, numerical analysis plays an essential role. In section 1.1, various processes occurring inside the furnace were discussed. These processes when added to the huge size of the blast furnace, make it an arduous task to visualise the furnace while it is in an operating condition. The blast furnace was considered a "black-box" where a way to "physically see" the furnace was by quenching it with cold nitrogen and dissecting the furnace [12]. Inspecting a quenched blast furnace allowed for identifying the different zones inside it along with the different ore/coke layers.To understand and learn about the blast furnace while it was operating, various probing devices and sensors[13–

15] were developed. These probes helped control the productivity rate of the furnace better than when operators in the past relied on experience and empirical means to maximise output. This gave a rise to phenomenological models being built alongside the empirical models. While the probing methods increased the productivity, many phenomenon remained unknown like the mechanics of raceway formation and the impact of varying inlet blast conditions. To understand and visualize the processes inside the furnace and predict the operation with respect to different operating parameters numerical models were being developed. The injection zone particularly in the blast furnace contains the harshest conditions, such as high temperature and high momentum transfer between the injectants and the porous bed combined with materials undergoing phase change in the raceway. This creates a challenge to accurately imagine in an experimental setup. Hence numerical models [16–21] provide a way to depict these phenomenon using theory and concepts from smaller experiments and other numerical models to build a robust simulation of various scales but with a common goal to incorporate theory into a complete model.

1.2 Motivation and Scope

In section 1.1, the extreme conditions and the complexity of the blast furnace was described. M2i, TUe and TATA Steel have been working on a project(hereby referred to as the "parent project") to understand the chemical, physical and overall characteristics of the raceway. The main goal is to learn about the conversion processes in the raceway to track dust particles and find ways to minimise production of dust.This will lead to development of better strategies to increase efficiency of the furnace and cut down CO_2 emissions as a result of better coke usage. As a result, a comprehensive model of the raceway is to be developed using DEM (Discrete element method, in which each particle has a set of governing equations that need to be solved, which increases the computational load but provides very accurate particle behaviour) and CFD to study the chemical and physical characteristics. A full-scale comprehensive thermophysical model of the raceway is to be built and the focus lies on capturing all the effects present in the furnace. This increases the computational cost to a high amount. To combat this, reduction techniques were looked into and two reduction techniques were chosen to be employed in the project.

1. Coarse-graining : To account for the large number of purverised coal particles, a reduction method is employed [22]. In this method, the interaction between a parcel of pulverised coal particles and a coke particle is being studied. This study is being carried out by another student in TUe.

2. FGM : It stands for Flamelet Generated Manifold. FGM is an reduction method for gaseous combustion wherein, instead of solving a large number of governing equations for species, only a few controlling variables are solved for and the rest of the variables are looked up from a pre-generated table. This ensures that lengthier time can be simulated, if not more in the same time as a detailed chemistry simulation [23].

The scope of this study is to employ FGM method for the combustion of pulverised coal particles. The solid coal particles are not considered here and will be employed at a later stage in a different project. The pulverised coal is replaced by the amount of volatile matter equivalent of pulverised coal in this study. Only the volatile matter combustion is of focus at present as the FGM method's acumen in gaseous phase combustion is being studied in this project. A devolatisation model has already been introduced in the parent project 1.2. It is therefore assumed that devolatisation reaction has been completed fully and the fuel is the product of the devolatisation reaction as employed by [24, 25]. This study deals with preparation of the FGM database/table using non-premixed 1-D flamelets which are solved in the specialised flame code Chem1d. The generated FGM table is used in OpenFOAM to compare with a full detailed chemistry simulation and the future use of this technique in scope of M2i's project.

1.3 Research Objectives

The research objectives of this study are as follows :

1. To generate an FGM database for gaseous pulverised coal combustion. Here gaseous pulverised coal combustion is equivalent to combustion of volatile matter in the pulverised coal particles.

2. To adapt an FGM model in the flow solver OpenFOAM, evaluate the performance of FGM model and simulate volatile matter combustion and compare with detailed chemistry combustion.

3. To explore a pathway for full gaseous combustion of pulverised coal combustion.

1.4 Report Outline

This section until now is provided with an aim to briefly discuss background knowledge about the important processes in the blast furnace and to define the scope of this study. Section 2 deals with the governing equations regarding combustion of volatile matter of coal released in the raceway. Firstly, the combustion process is defined as a 1-D non-premixed combustion case in a specialised flame solver Chem1d. Then the method of FGM is explained and the requirements for generating an FGM table are laid out. In section 3 the simulation settings corresponding to volatile matter combustion are explained and the process of making flamelets as well as the requisites for running an FGM method of combustion in OpenFOAM is described. In section 4, the generated FGM table is gauged by using it to generate 1D flamelets and comparing them with detailed chemistry simulations using Chem1d. Also the results of FGM simulation are compared to that of detailed chemistry simulation for a simple 2D counterflow setup in OpenFOAM. The final section 5 gives an interpretation of the results and a projection of this study towards the future.

Chapter 2

Theory

This section is an expansion of section 1.1, on the theory of gaseous combustion. This section will lay out the general equations for chemically reacting flows. Pulverised coal when injected in the tuyere undergoes devolatisation and releases volatile matter. The computation of these gaseous flow is very expensive owing to the large set of system of equations present for detailed chemistry simulation(the detailed reaction mechanism GRI-Mech has 53 species and 325 reactions). The fact that volatile matter combustion happens in a complex manner and includes a large number of different constituent substances in the volatile gas composition 1.1.2, demands a comprehensive model. Therefore as a result, the equations to be solved are very high. Hence a chemistry reduction method is introduced known as the Flamelet Generated Manifold (FGM) method [23]. FGM method reduces the large set of system of differential equations that define the flow, to a smaller number, which is also equal to the dimension of the lookup table from which other variables are directly looked up instead of solving from this database and placed in the flow solver. The required 1-D flamelets to generate FGM are generated by a specialised 1-D flame code known as Chem1d. Chem1d solves for flames in 1-D either using detailed chemistry mechanisms or by using an FGM table. More details about chem1d are discussed in 2.1.

Gaseous fuel combustion involves a large number of chemical reactions between a fuel and a oxidiser to release heat and light. These combustion processes can be broadly put in two types, premixed and non-premixed combustion. Premixed combustion takes place, when the gaseous fuel and gaseous oxidiser are introduced into the reactor pre-mixed in a specific ratio known as the air-to-fuel(AF) ratio. Non-premixed combustion on the other hand does not have the mixture of fuel. Two separate streams are introduced in the reactor which first mix and then combust. In a blast furnace, since the blast air and pulverised coal enter the raceway through two different streams [2], they follow the non-premixed regime. Combustion can be described by using a set of partial differential equations also knows as transport equations for fluid and chemical composition. The fluid flow is described by a set of conservation equations for mass, momentum and energy. Similarly, the chemical part of the combustion is described by the evolution of mass of species, finalized by an equation of state [26]. In the subsection 2.1, the functionality of Chem1d is introduced with the governing equations and various thermophysical models that chem1d makes use of. The next subsection 2.2 introduces the method of FGM and lays out its form of the governing equations and how it can be realised for a case of non-premixed gaseous volatile matter combustion. In section 2.3 a general layout of OpenFOAM 2.4.x is discussed briefly before discussing the implementation of generated FGM table with an FGM solver adapted from 'FGMFoam' by L. Ma. [27] in OpenFOAM.

Mass conservation is expressed by the continuity equation.

$$\frac{\partial \rho}{\partial t} + \nabla \cdot (\rho \vec{u}) = 0, \tag{2.1}$$

Here ρ denotes the density of the mixture, $\vec{u}$ is the 3-dimensional velocity vector of the fluid flow and t is the time.

Momentum conservation is expressed in the Navier-Stokes equation.

$$\frac{\partial (\rho \vec{u})}{\partial t} + \dot{\nabla} \cdot (\rho \vec{u}\vec{u}) + \dot{\nabla} \cdot \vec{\tau} + \nabla p - \rho \vec{g} = 0, \tag{2.2}$$

with $\vec{g}$ as the gravitational acceleration field, $\vec{\tau}$ being the stress tensor, and p being pressure. Here only the body forces pertaining to gravitational effect is considered.
The energy conservation equation is described in terms of specific enthalpy h.

$$\frac{\partial (\rho h)}{\partial t} + \nabla \cdot (\rho \vec{u} h) + \nabla \cdot \vec{q} + \vec{\tau} \colon (\nabla \vec{u}) - \frac{Dp}{Dt} = 0, \tag{2.3}$$

where $\vec{q}$ is the heat flux and the last two terms of the left hand side represent the production of enthalpy due to viscous dissipation and pressure variation, respectively. The varying composition of the reacting flow is described by the conservation of mass of all the chemically reacting species.

$$\frac{\partial \rho_i}{\partial t} + \nabla \cdot (\rho_i \vec{u_i}) = \dot{\omega}_i, \quad i \in [1, N_s] \tag{2.4}$$

with ρ_i the density of the species i, N_s is the total number of species, $\vec{u_i}$ is the velocity of the species i, and ω_i is the chemical source term of the species i, which is a representation of the formation or consumption of species by chemical reactions. Also since chemical reactions are conserved, the following equation can be drawn.

$$\sum_{i=1}^{N_s} \dot{\omega}_i = 0 \tag{2.5}$$

The velocity of species i can be split into two terms as,

$$\vec{u_i} = \vec{u} + \vec{U}_i, \tag{2.6}$$

where $\vec{u}$ is the bulk flow velocity and $\vec{U}_i$ is the diffusion velocity of species i. Using the following definition of species mass fraction Y_i, Eqn. 2.4 can be expanded as,

$$\frac{\partial(\rho Y_i)}{\partial t} + \nabla \cdot (\rho \vec{u} Y_i) + \nabla \cdot (\rho \vec{U_i} Y_i) = \dot{\omega}_i, \quad i \in [1, N_s] \tag{2.7}$$

This equation is represented in the general transport equation form clearly differentiating the diffusive, convective, temporal and source terms. The pressure is given by the perfect gas law as follows

$$p = \rho RT \sum_{i=1}^{N_s} \frac{Y_i}{M_i} \tag{2.8}$$

Here R and T are the universal gas constant and gas temperature respectively. The specific enthalpy is given by,

$$h = \sum_{i=1}^{N_s} Y_i / h_i \tag{2.9}$$

$$h_i(T) = h_i^0 + \int_{T^0}^{T} c_{p,i}(\tau) d\tau \tag{2.10}$$

where M_i, $c_{p,i}$, h_i, h_i^0 are respectively the molar mass, specific heat, specific enthalpy and the enthalpy of formation of species i.

2.1 Chem1d

Chem1d is a specialised flame solver in 1-D [28]. It solves the general set of conservation equations 2.1-2.10 using a detailed chemistry mechanism and various transport models 2.1.1 in one spatial direction. Chem1d uses an adaptive grid to rearrange and refine the grid according to the minimize local gradients. Important inputs to the Chem1d code are, the type of flame (premixed, non-premixed,..etc), fuel/oxidiser composition, temperature settings, amongst many others. An example of the settings file is shown in the appendix B. The output files of Chem1d contain profiles of species concentration, temperature, chemical source term to name a few along the 1D grid [29].

$$\frac{\partial \rho}{\partial t} + \frac{1}{\sigma} \frac{\partial}{\partial x}(\sigma \rho \vec{u_x}) = -\rho K \tag{2.11}$$

$$\frac{\partial \rho Y_i}{\partial t} + \frac{1}{\sigma} \frac{\partial}{\partial x}(\sigma \rho \vec{u_x} Y_i) = -\rho K Y_i - \frac{1}{\sigma} \frac{\partial}{\partial x}(\sigma \rho \vec{U_{X,i}} Y_i) + \omega_i \; i = 1, ..., N_s \tag{2.12}$$

$$\frac{\partial \rho h}{\partial t} + \frac{1}{\sigma}\frac{\partial}{\partial x}(\sigma \rho \vec{u_x} Y_i) = -\rho K Y_i - \frac{1}{\sigma}\frac{\partial}{\partial x}(\sigma \vec{q_x}) - \frac{\partial p^0}{\partial t} \tag{2.13}$$

with, x as the spatial coordinate, $\vec{u_x}$, $\vec{U_{x,i}}$, and $\vec{q_x}$, the x-components of $\vec{u}$, $\vec{U_i}$ and $\vec{q}$. The stretch rate K, and σ as surface area are introduced to model heatloss in other spatial coordinates and curvature [30]. The ρ is used from the ideal gas law, and the continuity equation can be used to find out the velocity $\vec{u_x}$, hence there is no need for a momentum equation and also due to the low mach assumption that is put in place. Which means that the pressure is now only a function of time.

$$p = p^0(t) \tag{2.14}$$

The transport and chemical quantities($\vec{U}$, $\vec{q}$, $\vec{\tau}$ and ω_i) are described in the next section.

2.1.1 Transport Model

The various species in a reacting flow, and their thermophysical properties are defined by the models in this section. These models will be used to solve equations 2.11-2.13. The species diffusion velocity $\vec{U_i}$ is found by the Stefan-Maxwell equations [31].

$$\vec{U_i} = \frac{1}{X_i \bar{M}} \sum_{j=1}^{N_s} M_{ij} D_{ij} \vec{d_j} - \frac{D_i^T}{\rho Y_i T} \nabla T \tag{2.15}$$

for low mach numbers effects of pressure and temperature gradient are omitted and the generalised diffusion coefficients D_{ij} and D_i^T are further given by,

$$\nabla X_i = \sum_{i=1}^{N_s} \frac{X_i X_j}{D_{ij}} (\vec{V_j} - \vec{V_i}) + ... \tag{2.16}$$

Here D_{ij} is the binary diffusion coefficient of species i into j. The viscous stress tensor can be modelled using Stoke's law assuming a newtonian fluid behaviour.

$$\vec{\tau} = \eta(\nabla \vec{u} + (\nabla \vec{u})^T - \frac{2}{3}(\nabla \cdot \vec{u})\vec{I}) \tag{2.17}$$

here, η is the dynamic viscosity of the mixture and $\vec{I}$ is a unit tensor.

The heat flux can be written as a form of the Fourier's law,

$$\vec{q} = -\frac{\lambda}{c_p} \nabla h + \frac{\nabla}{c_p} \sum_{i=1}^{N_s} (\frac{1}{Le_i} - 1) h_i \nabla Y_i \tag{2.18}$$

where λ is the thermal conductivity of the mixture and Le_i is the Lewis number which gives the ratio of mass and heat diffusion rate in a flame.

$$Le_i = \frac{\lambda}{\rho D_i c_p} \tag{2.19}$$

The dynamic viscosity is modelled by a mixture average approach stated by Wilke [32] as,

$$\eta = \sum_{i=1}^{N_s} \frac{\eta_i}{1 + X_i^{-1} \sum_{j=1,j\neg i}^{N_s} X_j \Phi_{ij}} \tag{2.20}$$

where

$$\Phi_{ij} = \frac{1}{\sqrt{8}}(1 + \frac{M_i}{M_j})^{-1/2}[1 + \frac{\eta_i}{\eta_j}^{1/2}(\frac{M_j}{M_i})^{1/4}]^2 \tag{2.21}$$

A further simplification of the transport models can be realised by using a constant Lewis number. Then the mass diffusion is given by,

$$\vec{U_i} = -\frac{D_{im}}{Y_i}\nabla Y_i, \tag{2.22}$$

where,

$$D_{im} = \frac{\lambda}{\rho Le_i c_p} \tag{2.23}$$

The species transport coefficients are drawn down from the kinetic theory of gases. They are tabulated as a function of temperature corresponding to the reaction mechanism that is being used. In this case the GRI Mech 3.0 which has 53 species and 325 reactions. These coefficients are stored in the 'Thermodynamcics file' of the mechanism package.

$$ln(\lambda_i) = \sum_{n=1}^{P} a_{n,i}(lnT)^{n-1} \; i = 1, ..., N_s \tag{2.24}$$

$$ln(\eta_i) = \sum_{n=1}^{P} b_{n,i}(lnT)^{n-1} \; i = 1, ..., N_s \tag{2.25}$$

$$ln(_ij) = \sum_{n=1}^{P} c_{n,ij}(lnT)^{n-1} \; i = 1, ..., N_s - 1, \; j = i + 1, ..., N_s \tag{2.26}$$

The definition of elemental mass fraction Z_j of an element j is given by,

$$Z_j = \sum_{j=1}^{N_s} (\frac{\alpha_{ji} M_j}{M_i}) Y_i = \sum_{i=1}^{N_s} w_{ji} Y_i \tag{2.27}$$

where N_e is the total number of elements, j is the number of elements ranging from [1,N_e], a_{ji} denotes the number of atoms of j in species i, M_i is the molar mass of species i. With this the conservation of species can be written as an elemental conservation equation.

$$\frac{\partial(\rho Z_j)}{\partial t} + \nabla \cdot (\rho \vec{u} Z_j) = \nabla \cdot (\frac{\lambda}{c_p} \nabla Z_j) + \nabla \dot{(} \frac{\lambda}{c_p} \sum_{j=1}^{N_s} (\frac{1}{Le_i} - 1) w_{ji} \nabla Y_i) \tag{2.28}$$

2.1.2 Chemistry Model

To define the chemical source term that is used in equation 2.4, a simple one step reaction is described as following. In the reaction

$$C + CO_2 \longrightarrow 2CO, \tag{2.29}$$

the reactants C and CO_2 react to form CO, the forward rate of reaction denoted by the subscript 'f' or the production rate of products is proportional to the concentrations of the reactants and is given as,

$$r_f = k_f [CO_2][C], \tag{2.30}$$

where the concentrations ($\rho Y_i / M_i$) of the species i is denoted by the square brackets. k_f is the reaction rate coefficient in the form of modified Arrhenius equation[33] is written as:

$$k_f = AT^\beta exp(\frac{-E_a}{RT}) \tag{2.31}$$

Here A and β are the pre-exponential factor and the exponent factor respectively. E_a is the activation energy for the reaction in J/mol^{-1}. The relationship between the species along time can be shown as,

$$\frac{d[CO]^2}{dt} = -\frac{d[CO_2]}{dt} = -\frac{d[C]}{dt} = r_f \tag{2.32}$$

Similarly, for the reverse reaction, if r_r is the reaction rate, the net reaction rate would be given by,

$$r = r_f - r_r = k_f [CO_2][C] - k_r [CO]^2 \tag{2.33}$$

The equillibrium constant k_{eq} is also defined as,

$$k_eq = \frac{k_f}{k_r}, \tag{2.34}$$

In general, a chemical reaction 'j' can be expressed as,

$$\sum_{i=1}^{N_s} v'_{ij} M_i \rightleftharpoons \sum_{i=1}^{N_s} v''_{ij} M_i, \tag{2.35}$$

where M_i is the species i, $v_{ij} = v''_{ij} v'_{ij}$ is the stoichiometric coefficient, indicating the number of molecules of species i taking part in the reaction j. N_s is the number of species. Similarly the general reaction rate of reaction j can be read as,

$$r_j = k_{fj} \prod_{i=1}^{N_s} [M_i]^{v'_{ij}} - k_{rj} \prod_{i=1}^{N_s} [M_i]^{v''_{ij}} \tag{2.36}$$

The total source term in the equation 2.4 has the effect of all the chemical reactions. that translates to ,

$$\omega_i = M_i \sum_{j=1}^{N_r} v_{ij} r_j, \tag{2.37}$$

where, N_r is the total number of reactions in the chemical process. The source term 2.37 along with the other chemical coefficients A, β and E_a are described in the comprehensive detailed chemistry mechanism in the CHEMKIN [34] format along with the thermophysical coefficients needed to calculate the diffusion, thermal conductivity and viscosity coefficient. This study will use the GRI 3.0 Mechanism which comprises of 53 species and a total of 325 unique reactions [35].

2.2 Flamelet Generated Manifold

In order to reduce computational costs, there are a few reduction techniques available to simplify the reaction kinetics. These techniques are based on the observation that a typical combustion system contains numerous chemical processes with a much smaller timescale than that of the flow timescale. Hence these fast processes can be decoupled in turn reducing the computational cost. Out of them, FGM (Flamelet Generated Manifold) was developed by J. van Ojien and P. de Goey [23] which is highly cost effective as compared to a detailed chemistry computation. In this method, a multi-dimensional flame is considered as an ensemble of 1-D flames. The implementation of this is as follows. Firstly a low-dimensional manifold is constructed in which all the thermo-chemical variables are stored. Since the 'manifold' is constructed using 1-D flames, it has the effects of diffusion better captured within it. Hence a better approximation than other reduction techniques such as the Intrinsic low-dimensional manifold (ILDM) or the computational singular pertubation (CSP) methods is observed in FGM. The solutions of these 1-D flames are stored in lookup tables which are referred with reference to one or few variables also known as the control variables. The chemical composition of the burnt mixture is determined by the conserved variables, while the combustion process from unburnt to burnt is described by the reactive controlling variables, which are called as progress variables [23].

Previous research on a single particle coal combustion in a laminar co-flow done by Vascellari et. al [36, 37] show that flamelet progress variable method show an excellent agreement with the reference solution when compared to a full transport and chemistry method. The numerical model by Reith et. al [38] also confirmed that the flamelet progress variable method can be used in a turbulent mixing layer to accurately predict the combustion process as the full transport and chemistry method. This gives the wide scope of the FGM method's robustness as a reduction technique and its possibilities.

For the FGM approach, the so called flamelet equation solutions are used to generate a low-dimensional manifold. This is a set of one dimensional flame adapted coordinate system equations that have been derived from the three dimensional governing equations 2.1-2.10. The equations in this method take up the following form if Lewis number is constant.

$$\frac{\partial \rho u_x}{\partial s} = 0 \tag{2.38}$$

$$\frac{\rho u_x Y_i}{\partial s} - \frac{\partial}{\partial s}(\frac{\lambda}{Le_i c_p}\frac{\partial Y_i}{\partial s}) = \dot{\omega}_i - \rho K Y_i \; i = 1, ..., N_s - 1, \tag{2.39}$$

$$\frac{\partial \rho u_x h}{\partial s} - \frac{\partial}{\partial s}(\frac{\lambda}{c_p}\frac{\partial h}{\partial s}) - \frac{\partial}{\partial s}(\frac{\lambda}{c_p}\sum_{i=1}^{N_s}(\frac{1}{Le_i} - 1)h_i\frac{\partial Y_i}{\partial s}) = 0 \tag{2.40}$$

These adapted flame coordinate version of equations 2.1,2.3,2.7 are known as the flamelet equations [23, 39]. s is the arc length perpendicular to the flame surface. Since we are considering non-premixed flamelets in this scenario, which means that there are two seperate streams for the fuel and oxidiser into the calculation domain, another equation of a transported mixture fraction is also solved. This ranges from zero to unity from the oxidiser side to the fuel side. The equation is as follows,

$$\frac{\rho u_x Z}{\partial s} - \frac{\partial}{\partial s}(\frac{\lambda}{Le_i c_p}\frac{\partial Z}{\partial s}) = -\rho K Z \tag{2.41}$$

A constant Lewis number model($\mathrm{Le_i} = 1$) is adapted which simplifies the equation. The above set of equations 2.38-2.41 are solved with suitable models from section 2.1 for a counterflow planar diffusion flame. The stretch field K(s) which is derived from the momentum equation [30] in the traverse direction is used in equation 2.39.

$$\frac{\partial(\rho u K)}{\partial x} = \frac{\partial}{\partial x}(\eta \frac{\partial K}{\partial x}) - 2\rho K^2 + \rho a^2 \tag{2.42}$$

where, η is the dynamic viscosity and a represents the strain rate $[s^{-1}]$ applied at the oxidiser side. The strain rate is varied and a flamelet solution for every strain rate is saved until the extinction strain rate. After this extinction of flame, transient solution of flamelet is calculated iteratively while keeping the strain rate at the extinction level. This is done until the flame completely quenches (flame does not ignite). To these above procured flamelet solutions, a reaction progress variable is chosen to parametrize this set of solutions. This progress variable is generally taken as the linear combination of species. So finally, to parametrize the flamelets in this study, the following are taken as control variables, and since two variables are taken to parametrize the database, it is a 2-D FGM database

$$CV_1 = \Phi_1 = Z \tag{2.43}$$

$$CV_2 = \Phi_2 = \alpha_1 Y_1 + \alpha_2 Y_2 + \alpha_3 Y_3 + \; ..... \; + \alpha_J Y_J. \tag{2.44}$$

Where $\alpha_1, \alpha_2, \alpha_3, \alpha_J$, are weight factors of Y_1, Y_2, Y_3 and Y_J respectively. CV_2 is also known as the progress variable.

The database is discretized in an equidistant grid as a function of two control variables. These two scalars are solved in the flow solver OpenFOAM shown as shown in section 2.3. The rest of the variables stored in the database are looked up via a bilinear interpolation method. This reduces the computational load greatly because, instead of a multitude of species transport equations, only two transport equations apart from the momentum equations are solved which are (for unity Lewis number),

$$\frac{d(\rho\Phi_1)}{dt} + \vec{\nabla}(\rho\vec{u}\Phi_1) = \vec{\nabla}(\frac{\lambda}{c_p}\vec{\nabla}\Phi_1) \tag{2.45}$$

$$\frac{d(\rho\Phi_2)}{dt} + \vec{\nabla}(\rho\vec{u}\Phi_2) = \vec{\nabla}(\frac{\lambda}{c_p}\vec{\nabla}\Phi_2) + \omega_{\Phi_2} \tag{2.46}$$

where, Φ_1 and Φ_2 are transported Z and Progress Variable, respectively and ω_{Φ_2} is the source (production or consumption of the species selected as the progress variable).

2.3 OpenFOAM - FGM coupling

This section aims to employ the transport equations discussed in 2.2 in a standard CFD solver. OoenFOAM is chosen as the CFD solver to simulate gaseous combustion using FGM and with a detailed chemistry mechanism and to compare the results of FGM and detailed chemistry.

OpenFOAM is an open source CFD code written in C++ that is extensively used in academia/industry for its open and customizable structure. OpenFOAM stands for '**Open**-source **F**ield **O**peration **A**nd **M**anipulation'. It is a toolbox written in C++ which contains a set of solvers and utilities as modular applications to solve a specific set of instructions (for example, reactingFoam is a solver to solve transient, compressible, turbulent, chemically reactive flows using a finite volume method). The complete OpenFOAM package consists utilities starting from generating a mesh and up to preparing files for post processing. A highly structured code for different functionalities (for example, numerical methods, mesh readers, thermophysical models, ..etc) are categorized, and compiled seperately. Then an executable application is linked to these compiled libraries accordingly to form a solver for a specific use-case [40].
Some of the main features and drawbacks of OpenFOAM are :

1. Since it is an open source project, its costs zero money to install, use and customise it.
2. It employs a very modular code design which is feasible to extend and make customised solvers.
3. It has a fully documented source code and a friendly syntax for complex differential equations.
4. Nevertheless, the programmer's guide lacks sufficient information about OpenFOAM development, making it a difficult task to begin quickly with customising solvers.
5. It lacks a Graphical User Interface, making it very difficult for newcomers.
6. It doesn't come with a post-processor but is mainly used in conjunction with Paraview.

The OpenFOAM 2.4.x software directory and the simulation case directory are shown in figures 2.1, 2.2.

In figure 2.1, the folder 'applications' contains the compiled executables in the solvers(for specific physical/chemical problem) and in the utility (auxiliary executables to aid functionality to the main solver) folder. The source code is included in the 'src' folder. Many tutorials for specific cases are available in the 'tutorials' folder which most of the time is a good place to start a case. In figure 2.2, the OpenFOAM case directory is highlighted, this has 3 main folders, namely 'constant', 'system', '0' (time directories). The system folder contains the settings for configuring the case, here the discretisation schemes, solver algorithms, tolerances and the general control of the solver is set like the start-time, end-time of the solution. In the constant folder, the different thermophysical and chemical properties are defined for the case, also here the mesh is defined, which can be made using the OpenFOAM's meshing utility blockMesh, or a mesh made in a third party software. The time directories start with an initial time folder '0', which has the initial condition's for the different vectors and scalars in the simulation. Any subsequent time directory (for example: ‘0.01’, ‘0.02‘,.. etc) are the solution fields at the specified time defined in the ’controlDict’ file of the ’system’ folder.

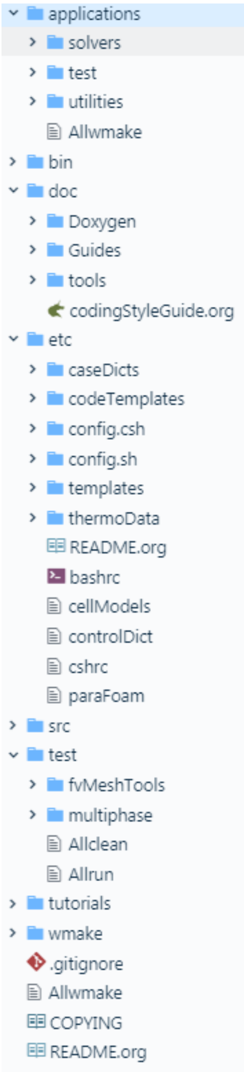


Figure 2.1: OpenFOAM software directory

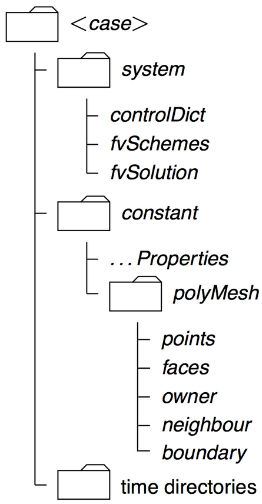


Figure 2.2: OpenFOAM case directory

2.3.1 reactingFoam Solver

To solve chemically reactive flows in OpenFOAM, reactingFoam is one such solver to simulate transient, compressible combustion using a PIMPLE algorithm[1] (which combines the SIMPLE algorithm [41] and the PISO algorithm [42] to solve the pressure-velocity coupled equations.) A simplified structure of reactingFoam is given below

```

    pimpleControl pimple(mesh);

```

[1]Simplified PIMPLE algorith flowchart in Appendix section D

```
...

#include rhoEqn.H         //Calculate density
while (pimple.loop())
{
    #include UEqn.H       //Calculate the U/p field
    #include YEqn.H       //Species transport equation with reaction source term
    #include EEqn.H       //T from chemical reaction enthalpy lookup

    // --- Pressure corrector loop
    while (pimple.correct())
    {
        #include pEqn.H   //PISO loop for p calculation
    }

    if (pimple.turbCorr())
    {
        turbulence->correct(); //turbulence modelling
    }
}

runTime.write();            //write fields

...
```

The simplified YEqn.H which represents eqn 2.7 is given by :

```

...

volScalarField& Yi = Y[i];

fvScalarMatrix YiEqn
(
    fvm::ddt(rho, Yi)
    + mvConvection->fvmDiv(phi, Yi)
    - fvm::laplacian(turbulence->muEff(), Yi)
    ==
     reaction->R(Yi)                  //chemical source term for species
    + fvOptions(rho, Yi)
);
...
```

The source term R(Yi) is calculated in the combustion model. Some of the combustion models present to be used in reactingFoam are laminar, infinitelyFastChemistry, singleStepCombustion,... etc. Other relavant thermophysical properties in scope of this study are discussed in section 3.

2.3.2 FGMFoam Solver

The solver used to implement FGM on OpenFOAM is known as the FGMFoam solver which is an adaptation of the reactingFoam solver by changing the equations to be solved and adding an FGM look up method to get the thermophysical properties directly from the table instead of calculating them as in the reactingFoam. FGMFoam was implemented by Likun Ma [27] in OpenFOAM v.2.3.1. The main differences in the way the equations are solved are showed in this subsection. The lookup method is a bilinear inetrpolation D method that is employed by attaching the FGM lookup code FanzyFGM [43–45] for 2D-FGM database.

```

...
#include rhoEqn.H
...
// —— Pressure—velocity PIMPLE corrector loop
while (pimple.loop())
{
    #include UEqn.H
    #include ZEqn.H      //Mixture—fraction transport equation
    #include PVEqn.H     //Progress—Variable transport equation

        // Pressure corrector loop
    while (pimple.correct())
    {
    #include pEqn.H
    }

    turbulence—>correct();
...
}
...
```

The PVEqn is simplified here similar to 2.46:

```

...
fvScalarMatrix PVEqn
(
        (
```

```
    fvm::ddt(rho, PV)
        + fvm::div(phi, PV)
    - fvm::laplacian(thermo.alpha()+turbulence->alphat(), PV)
    ==
    thermo.sourcePV()           // source term looked up in the FGM database
    )
);
max(PV).value() = 1.0;          //bounds the progress variable between 0 and 1
min(PV).value() = 0.0;
...
```

The ZEqn is simplified here similar to 2.45:

```

...
fvScalarMatrix ZEqn
(
(
 fvm::ddt(rho, Z)
+ fvm::div(phi, Z)
- fvm::laplacian(alpha+turbulence->alphat(), Z)
)
);
...
```

A flowchart of FGMFoam is is drawn here for visualisation of the solver algorithm.

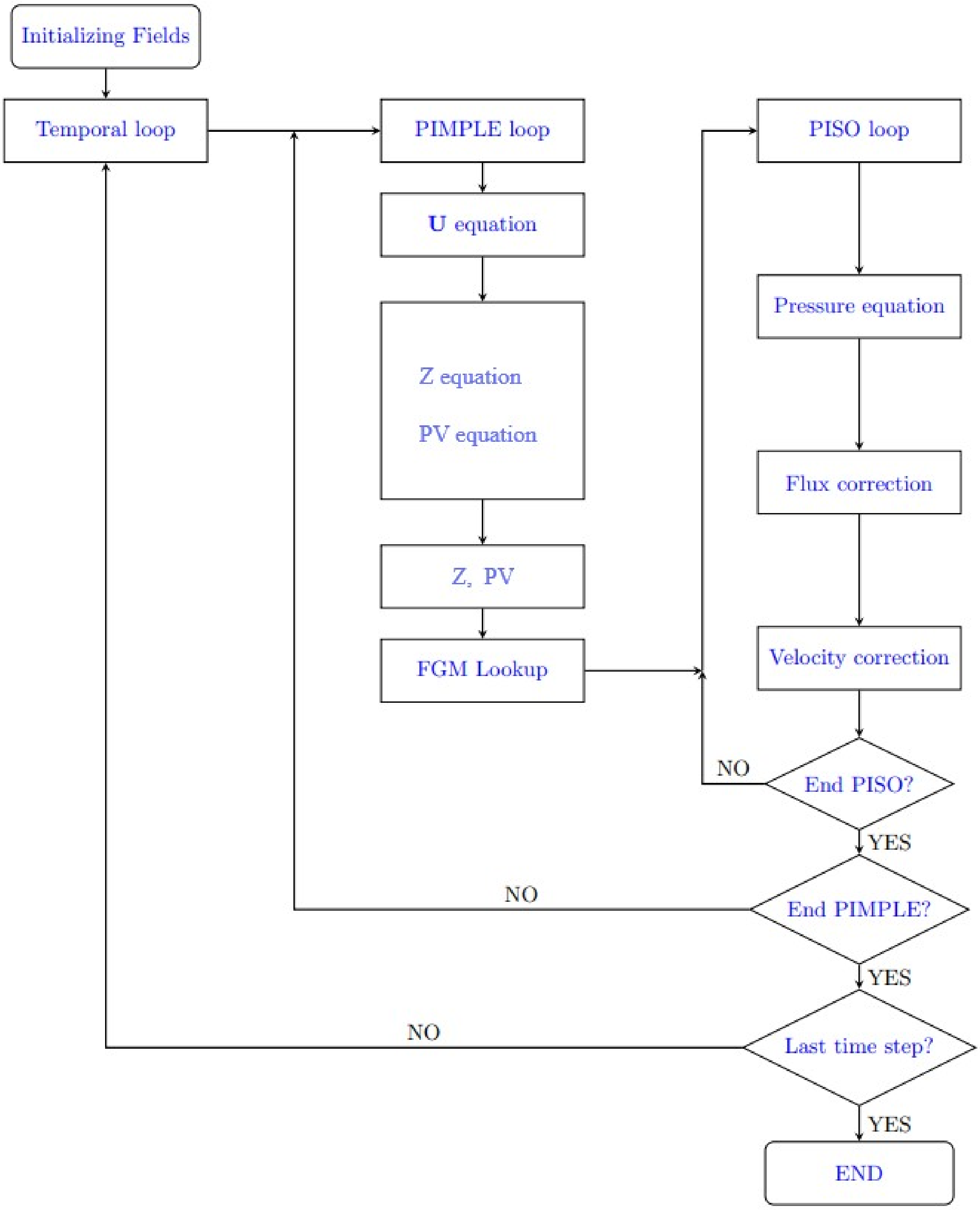


Figure 2.3: Solver algorithm FGMFoam [27]

Chapter 3

Methodology

In this section, the procedure of generating 1-D flamelets in Chem1D, and the procedure of FGM table creation are described. This FGM table is used to simulate a 2D non-premixed combustion in OpenFOAM to test the implementation of the solvers disscussed in 2.3. All other relevant simulation settings are laid down and explained.

3.1 Generation of 1-D Flamelets and a 2-D FGM Table

Chem1d runs on the terminal in Windown/Linux. There are two files in which simulation settings can be changed, namely the 'defaults.csf' file and the 'settings.csf' file. The simulation settings are as follows for generating 1-D flamelets.

	CH4	C2H2	C2H4	CO	H2	N2	CO2	H2O	O2	Temp (K)
Fuel	0.1158	0.5134	0.0706	0.0418	0.1001	0.0274	0.0209	0.1094	-	1173
Air	-	-	-	-	-	0.79	-	-	0.21	1473

The values for fuel(gaseous volatile matter) are derived from the coal at Newlands bituminous coalfired experiments as described in [46, 47].

MODEL_FLAMETYPE	GRID_NUMBEROFPOINTS	MODEL_TRANSPORT
COUNTERFLOW	300	LEWIS

An example of an 1-D flamelet is shown in figure 3.1. The x-axis shows the domain of the simulation extending from -6 cm where fuel enters and ends at 6 cm where air enters. Directional arrows also have been made to aid the understanding of the flow direction of fuel and air. At the centre, where both streams meet, chemical reaction takes place. In figure 3.2, different constituents of fuel and air are seen.The x-axis is intentionally zoomed into see the gradients in detail because the reaction zone is very thin.

Similarly, 1-D flamelets are generated from a = 0.1 s^{-1} to a_q, where a steady flamelet fails to converge and transient simulation is required to further simulate the flames. a_q is found to be 179000 s^{-1}.For this change the setting of 'MODEL_SIMULATIONTYPE' to 'TIMEDEPENDANT' while keeping the strain rate at the quenching value 'a_q'. Figure shows the quenching of flame even further in time-dependant simulation, and since chem1d has no automatic cutoff, monitoring this graph which is saved as 'fort.67' while running time-dependant or unsteady simulations is very important. Once the graph stabilises at a much lower temperature than it was when it is ignited, then the flame has been quenched.

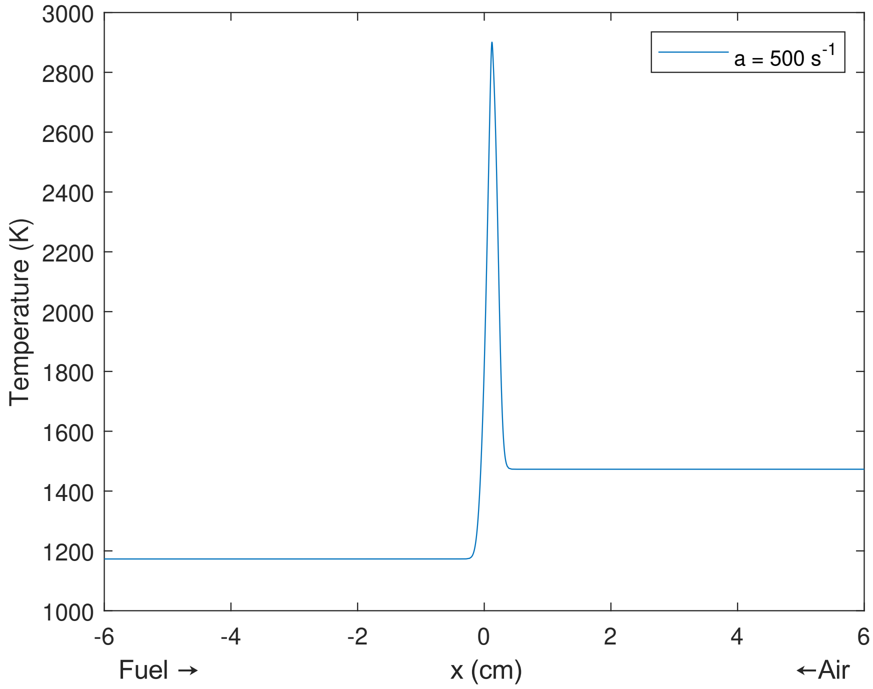


Figure 3.1: Temperaute plot at strain rate 500 s^{-1}

Figure 3.3 shows the temperature vs. mixture fraction plots for various strain rates 'a'. The red line closer to the x-axis in the figure is the pure mixing line, the strain rate is so high that, it doesn't allow combustion to sustain. An arrow is also drawn to show the trend of increasing strain rate. Figure 3.4 shows the falling trend of temperature from very high up top. This helps one decide on when to stop the unsteady or time-dependant simulation as there is no 'switch' for itself to turn off.

3.2 Generating 2-D FGM Tables

The solved flamelet equations are to be put up in a table or a database, for this, an equidistant rectilinear grid with axes equal to the number of controlling variables present, that is two in this study. Next, comes the choice of progress variable, and is chosen in this case as

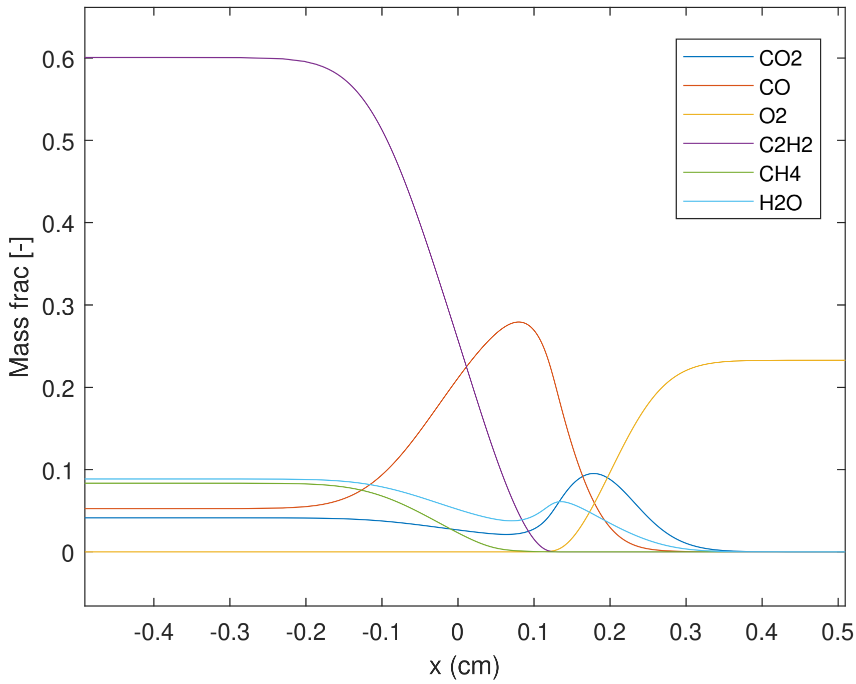


Figure 3.2: Mass fraction of different species at strain rate 500 s^{-1}

CO2/H2O/H2/CO, and their respective weights are 1/1/1/1.5.

$$CV_1 = Z, \tag{3.1}$$

$$CV_2 = PV = 1.15 * CO + CO_2 + H_2O + H_2 \tag{3.2}$$

The database is spread out on a 2D grid of size 300 x 300. The database is validated against the detail chemistry solutions using Chem1d for various strain rates to confirm accuracy and performance. Chem1d has an FGM mode where it reads the FGM table and solves the equations 2.45 and 2.46 and returns the tabulated variables (in this case temperature) as seen in figure 4.3.

3.3 OpenFOAM

To test the FGM table and a comparison is made with the standard reactingFoam solver of OpenFOAM in terms of accuracy and computation time. A simplified geometry is used to study and test the implementation of the FGMFoam solver. A larger, realistic coal burner was not chosen because of time constraints. The configuration is a 2-D counter flow setup to simulate non-premixed combustion with the domain having the following dimensions - (0.05 0.02 0.02)m. Since OpenFOAM always has a 3-D mesh, the number of cells in the z-direction are set to be 1 so it transforms to a 2-D case. The domain has 300 cells in x-direction, 40 cells in y-direction and 1 cell in z-direction. The box has 4 types of boundary types : fuel, air, outlet and empty.

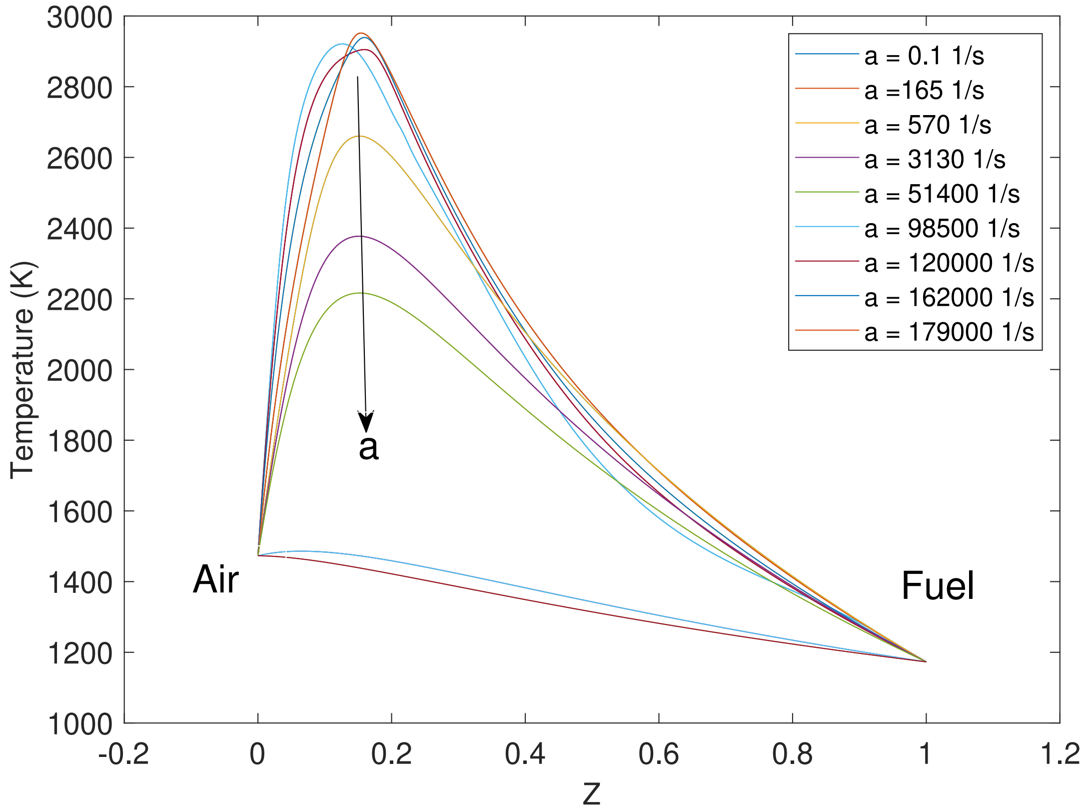


Figure 3.3: Temperature vs. Mixture Fraction for increasing strain rates

This mesh is made with the utility 'blockMesh' within OpenFOAM. The boundary conditions are given in the table 3.1, 3.2 and 3.3. The mesh is displayed in figure 3.5. The same mesh is used for simulating non-premixed flames using the reactingFoam solver and FGMFoam solver.

The boundary conditions as used in the reactingFoam solver are :

	U (m/s)	p (N/m^2)	T (K)	Y$_{default}$
fuel	(0.1,0,0)	zG	1173	0
air	(-0.1,0,0)	zG	1473	0
outlet	pressureInletOutlet (internalField)	totalPressure 1e5	iO(1173)	iO(0)
empty		empty		

Table 3.1: Boundary conditions - common

- **pressureInletOutlet (internalField) -** Outflow velocity is set to zero gradient condition, and inflow is set to the specified value (in this case its the value of internal field of the patch on the boundary)

- **zG** - zero gradient condition

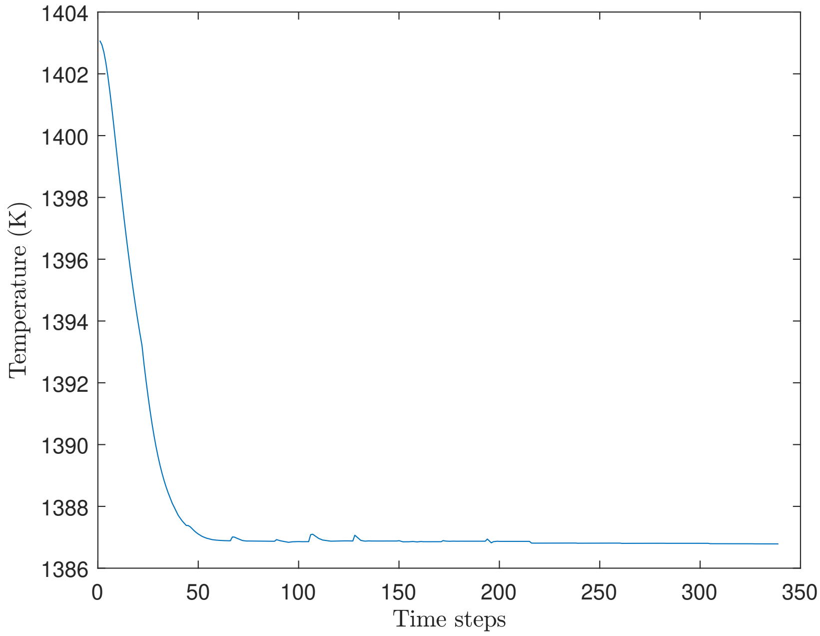


Figure 3.4: Monitoring of unsteady flames on Chem1d

	C2H2	C2H4	CH4	CO	CO2	H2	H2O	N2	O2
fuel	0.5134	0.0706	0.1158	0.0418	0.0209	0.1001	0.1094	0	0
air	0	0	0	0	0	0	0	0.79	0.21
outlet				inletOutlet (0)					
empty				empty					

Table 3.2: Boundary conditions - reactingFoam specific

- **iO(0)/inletOutlet(0) -** sets the outflow as a zero gradient condition and for inflow it sets the value specified (in this case 0).
- **empty -** this boundary condition indicated the solver to not solve the equations whose direction is normal to this boundary patch (in this case, the normal patch to this boundary is the z-direction, hence the governing equations are not solved for z-direction).

The case directory for the reactingFoam and the FGMFoam cases are expanded[1]. They have three folders, "0" which contains the boundary conditions, "constant" folder which contains the different thermophysical properties that are being used and finally the "system" folder which configures the numerical methods and the solution controls for the case.

1. **0 :** This folder contains files with the boundary/initial conditions as specified in tables 3.1, 3.2 and 3.3 according to the solver in an OpenFOAM format.
2. **constant :** This folder contains thermophysical data required for the simulation.

[1]exact boundary condition files are present in the appendix

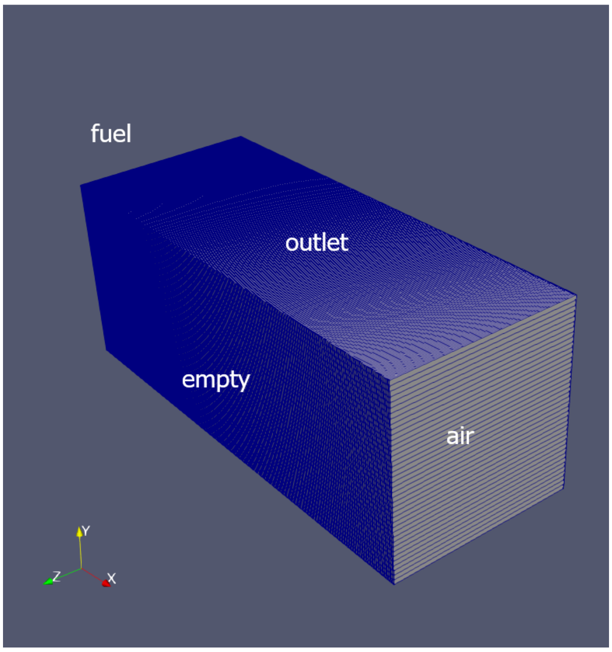


Figure 3.5: OpenFOAM mesh for counter-flow combustion

	Z	PV	source$_{PV}$
fuel	1	0	0
air	0	0	0
outlet	inletOutlet (0)		
empty	empty		

Table 3.3: Boundary conditions - FGM specific

- **chemistryProperties :** An option to turn on/off chemistry reactions is present along with the type of ODE solver to be used.
- **combustionProperties :** A combustion model is to be specified in this file. A laminar combustion model is selected here.
- **g :** The gravitational acceleration value is specified in this file.
- **reactions :** This file contains contains the reaction mechanism in OpenFOAM format which can be transformed from CHEMKIN format using the 'chemkinToFoam' utility.
- **thermo.compressiblegas :** It contains the thermophysical coefficients that are used to compute transport and thermodynamic variables according to the models specified in the 'thermophysicalModels' file in this same directory.
- **thermophysicalModels :** It contains the equations of state and transport model used. It also specifies the form of energy used (enthalpy or internal energy).
- **turbulenceProperties :** It contains the turbulence model that is used, laminar in this case.

- **fgmproperties**[2] **:** It contains the indices of variables corresponding to the FGM table.
- **database.fgm :** This is the FGM table generated as described in section 3.1.

3. **system :** This folder contains configuration files that describe the solution process parameters. The files are as follows :

   - **ControlDict :** This file contains the control parameters for the simulation. It has the start/end time, the write interval of output files.
   - **fvSolution :** This file contains the controls for the solver algorithm and the residual tolerences are set.
   - **fvSchemes :** This file contains the discretisation schemes for time derivative, gradient terms, divergence terms, laplacian terms and surface normal gradient.
   - **decomposePar :** This file enables parallelisation of simulation. The number of sub-domains are defined here which will be solved by different processors.

The counter-flow simulation is carried on using both the solvers for the already defined boundary conditions for 1 second and then later compared.

[2]only for FGMFoam

Chapter 4

Results

The FGM table is depicted here as a function of two control variables defined in section 3.2. Figure 4.1 displays the values of source term of CV_2 as a function of CV_1 and CV_2 and figure 4.2 displays the temperature data in the table as a function of the two controlling variables. All the required variables are stored in such a manner which will be looked up. Figure 4.3 shows

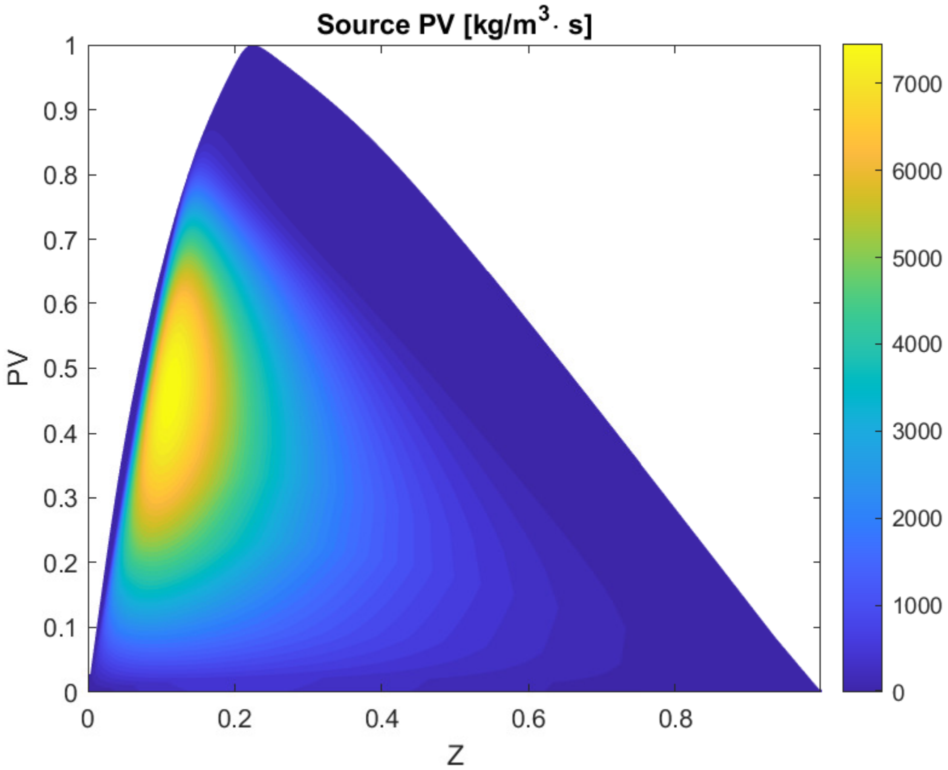


Figure 4.1: Source of PV as a function of Z and PV

the 1-D temperature profiles using FGM profiles compared to detailed chemistry simulations on Chem1d. For the strain rate of 1000 1/s, the FGM tends to differ just a bit from the detailed chemistry case at approximately x= 0.2 cm. Nevertheless, for the rest of the cases it confirms perfectly to the detailed chemistry temperature profiles. This shows that the FGM generated

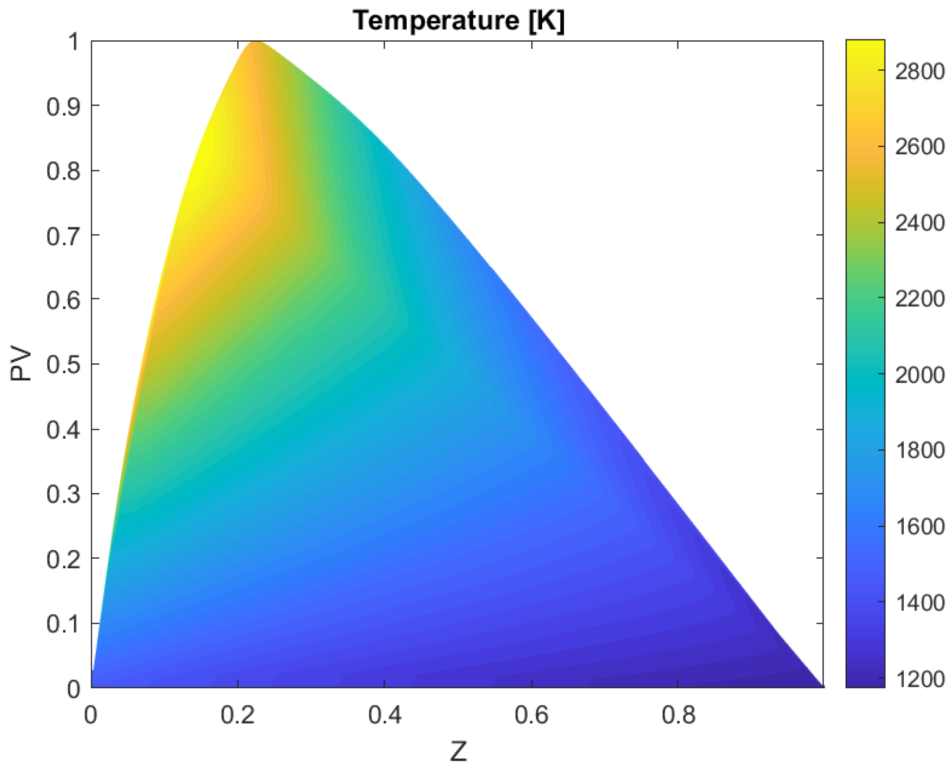


Figure 4.2: Temperature as a function of Z and PV

in section 3.2 captures the whole range of flamelets. This is an indicator of an good FGM table and indeed it can be further used as a lookup table for higher dimension simulations. The figures 4.6-4.13 are the contours of temperature profiles at xy-plane at z=0, for detailed chemistry and FGM simulation for the times 0.1, 0.5, 1 and 2 seconds. The detailed simulation case are seen in figures 4.6, 4.8, 4.10 and 4.12. In detailed chemistry a clear evolution of a temperature profile is seen, from the ends to the reaction zone and it converges. Meanwhile in the FGM case, the temperature profile instantly develops but fluctuates. The FGM table's accuracy is already demostrated in 4.3. Hence an error in linking of the FGMLookup might be a possible case for the error.

Figures 4.14 and 4.15 plot the temperature profiles at the line shown in the fig 4.5. The computation time comparision is shown in the figure 4.4. Detailed chemistry simulation takes close to 5 times more time to simulate 1 second of combustion. The trend of execution time is also upwards and one can imagine that it will be exponentially slower than FGM for even longer simulated times.

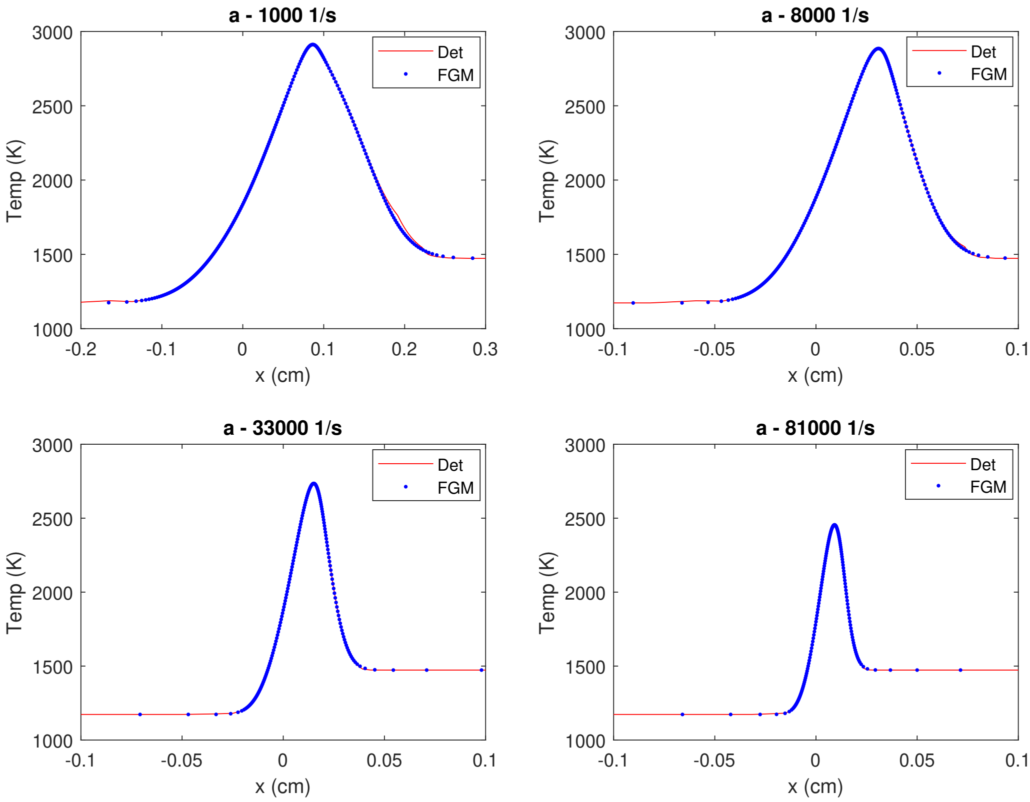


Figure 4.3: Temperature comparison Chem1d - Detailed vs FGM

The figures 4.16 gives the source of CV_2(PV) for t = 0.1 - 2 seconds.

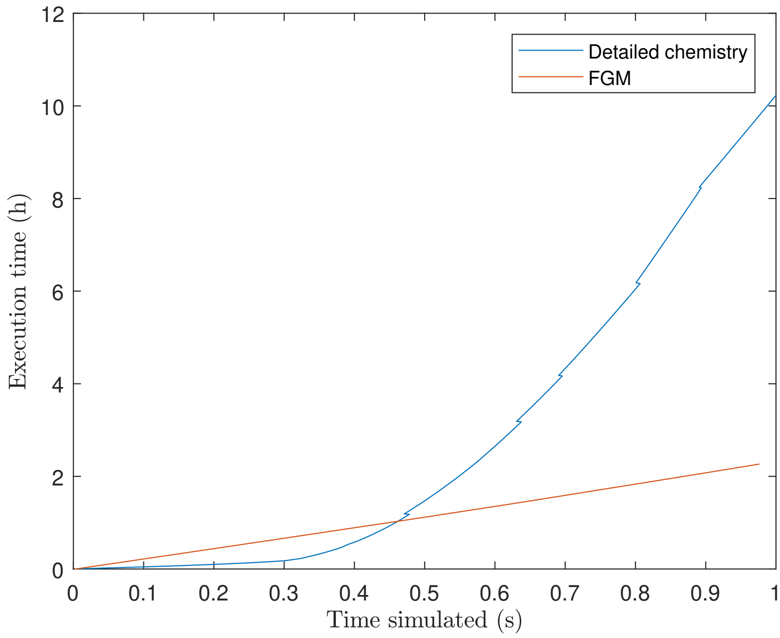


Figure 4.4: Time comparison of detailed chemistry simulation Vs. FGM on OpenFOAM

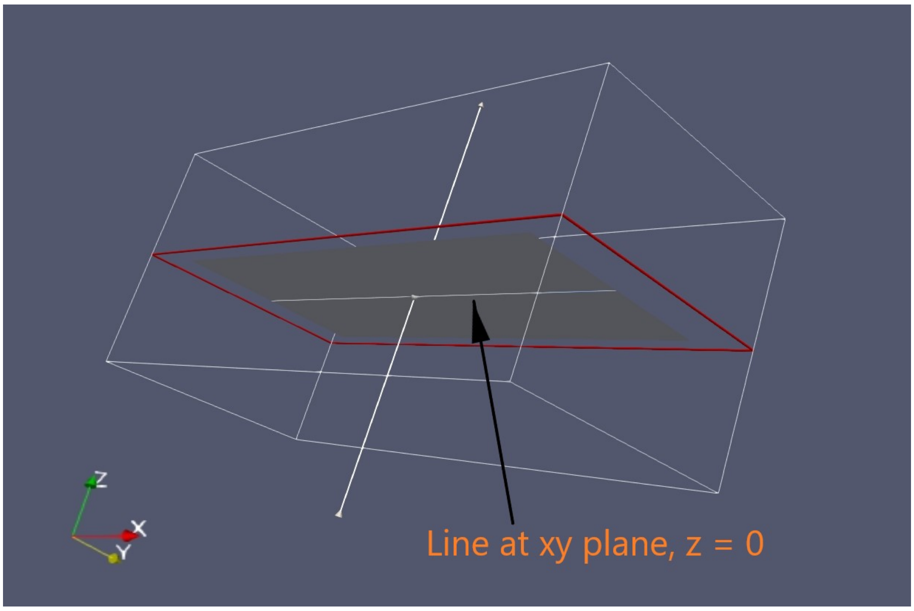


Figure 4.5: XY plane, z = 0, Temperature profile plotted at the line shown.

Figure 4.6: Detailed chemistry simulation at 0.1 sec

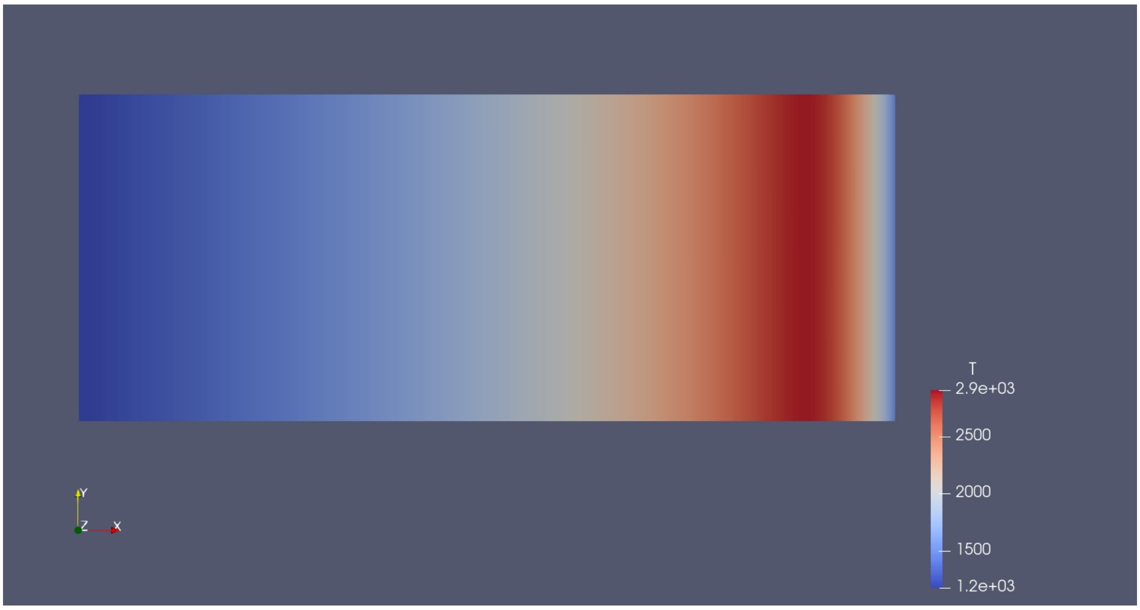


Figure 4.7: FGM simulation at 0.1 sec

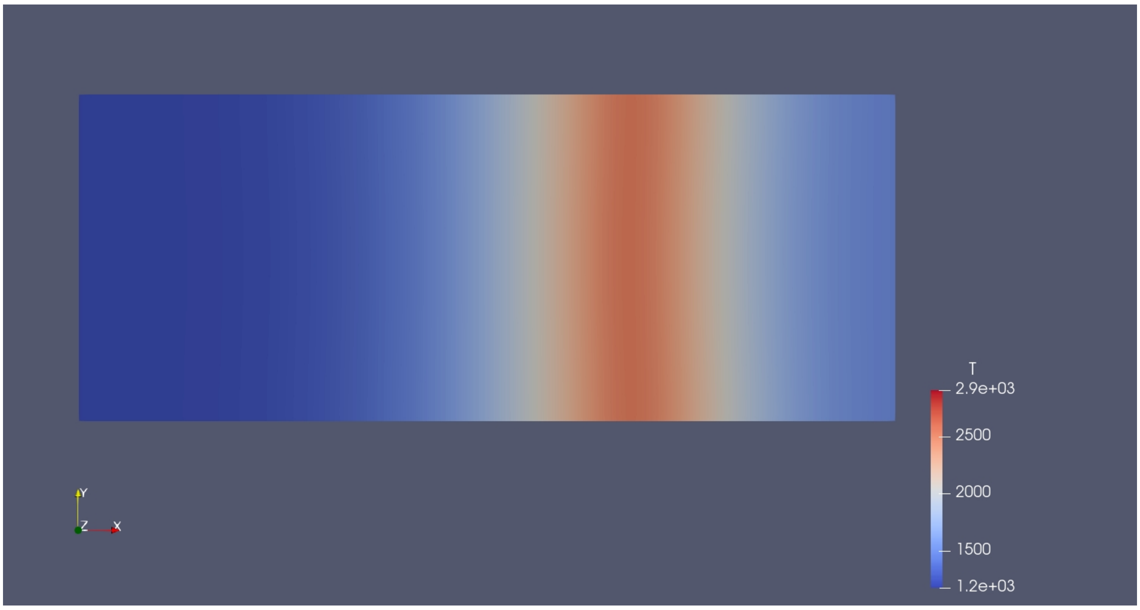


Figure 4.8: Detailed chemistry simulation at 0.5 sec

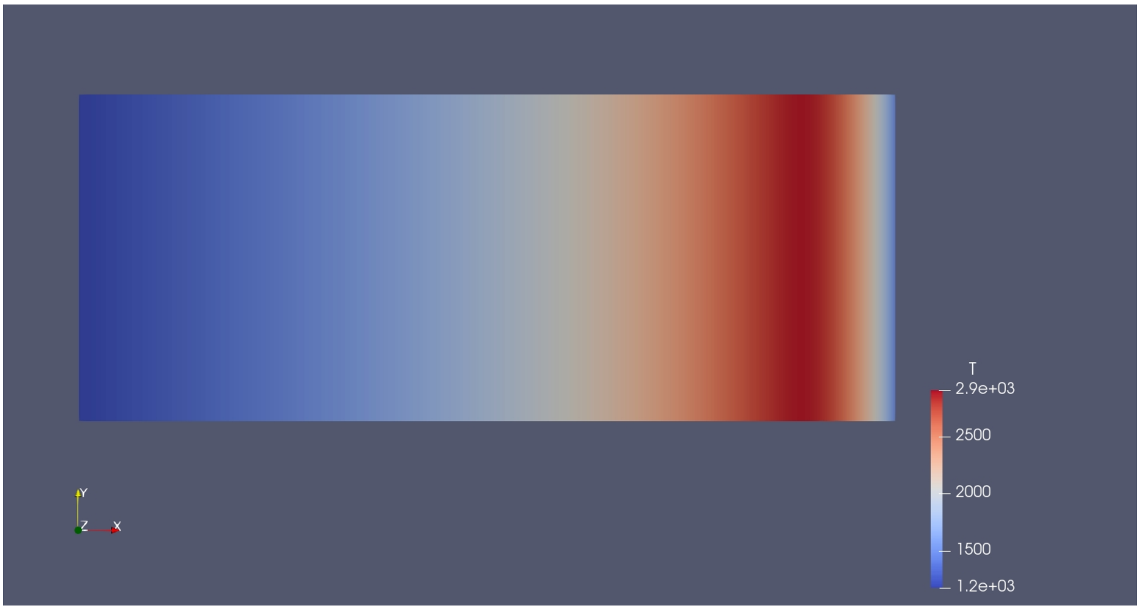


Figure 4.9: FGM simulation at 0.5 sec

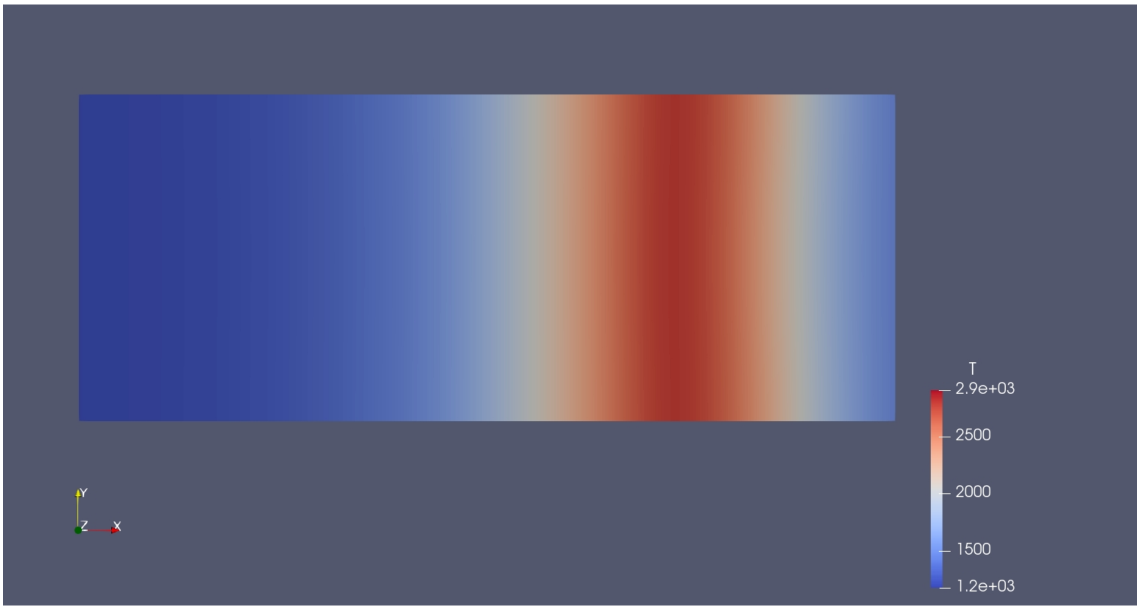


Figure 4.10: Detailed chemistry simulation at 1 sec

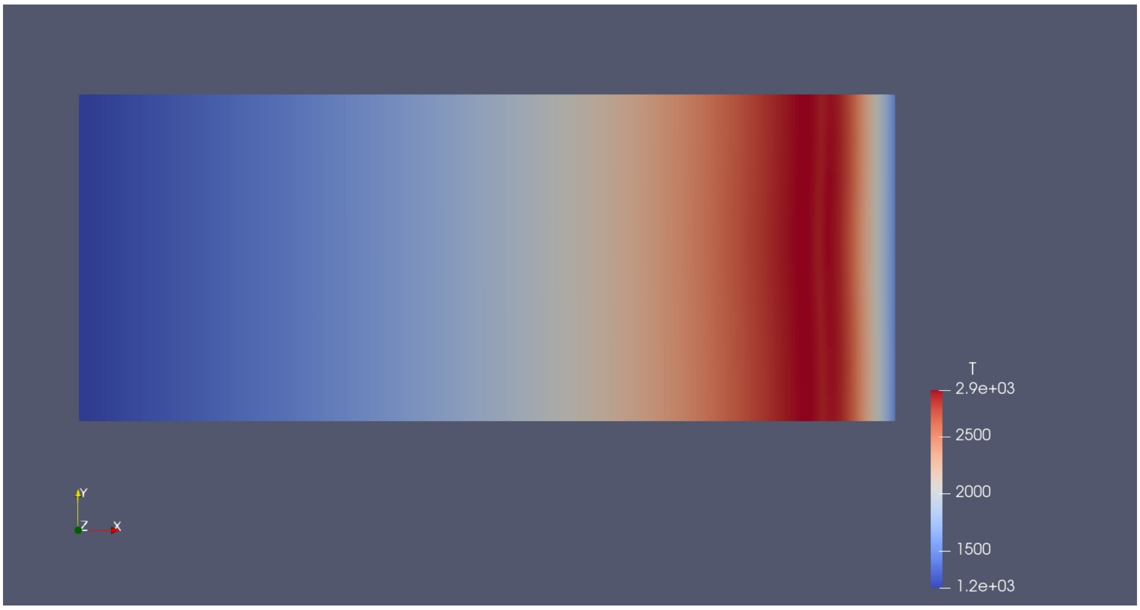


Figure 4.11: FGM simulation at 1 sec

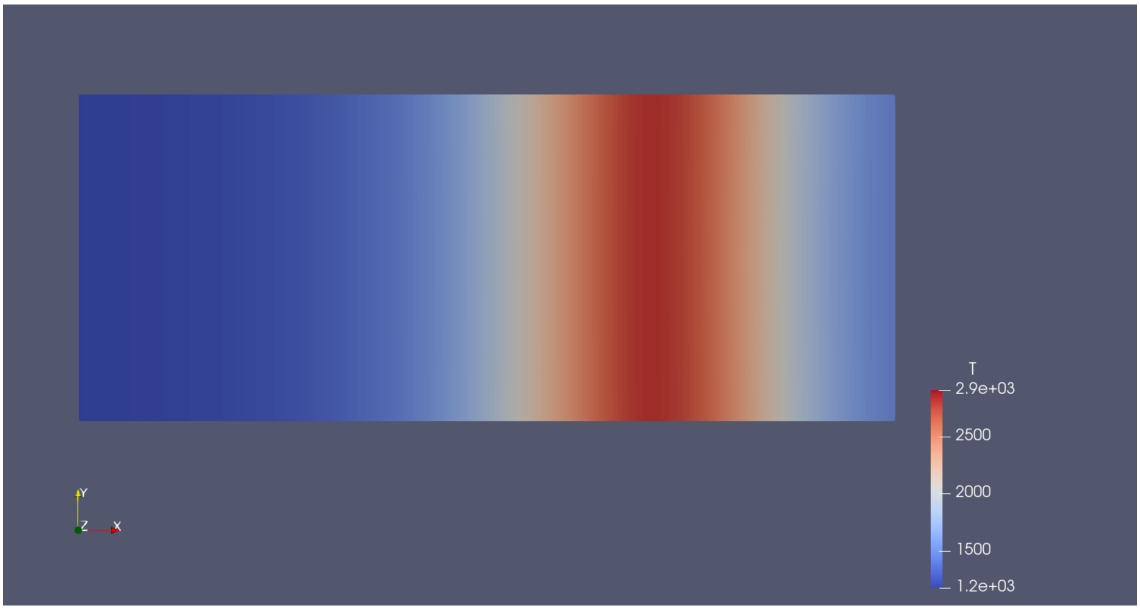


Figure 4.12: Detailed chemistry simulation at 2 sec

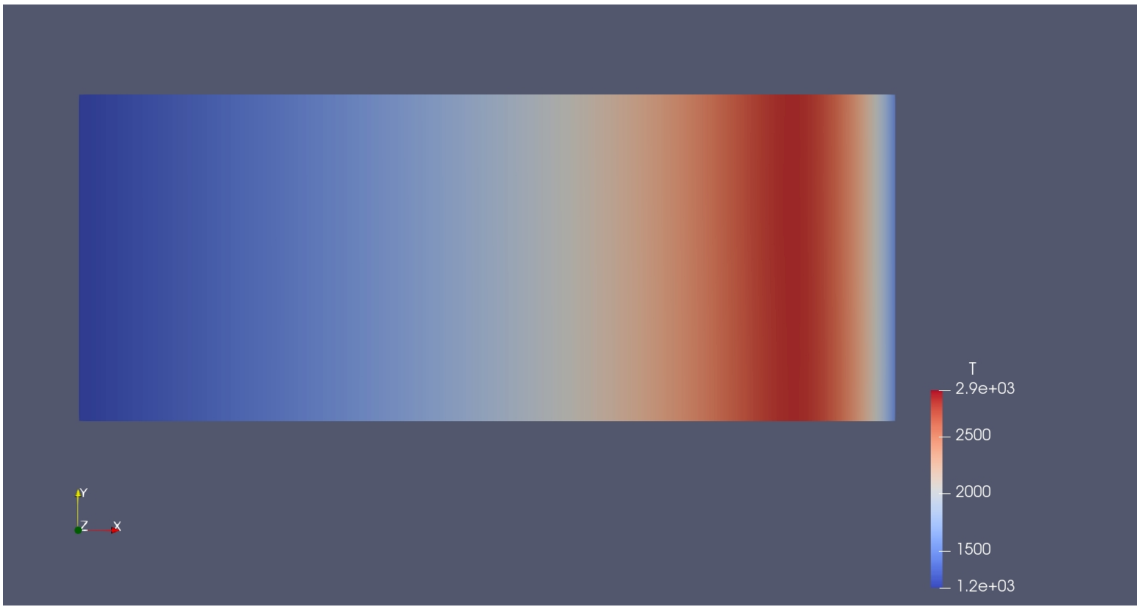
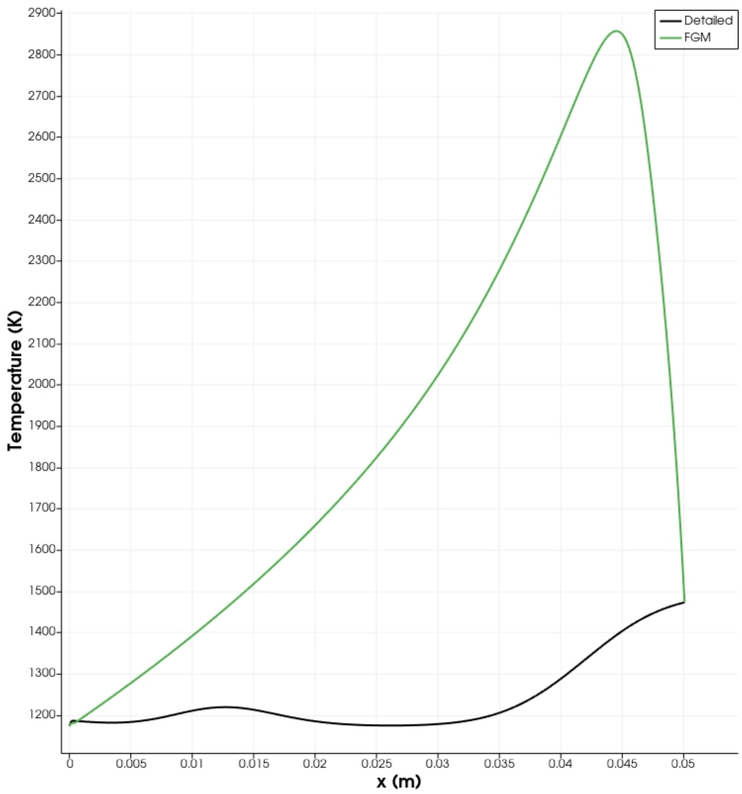
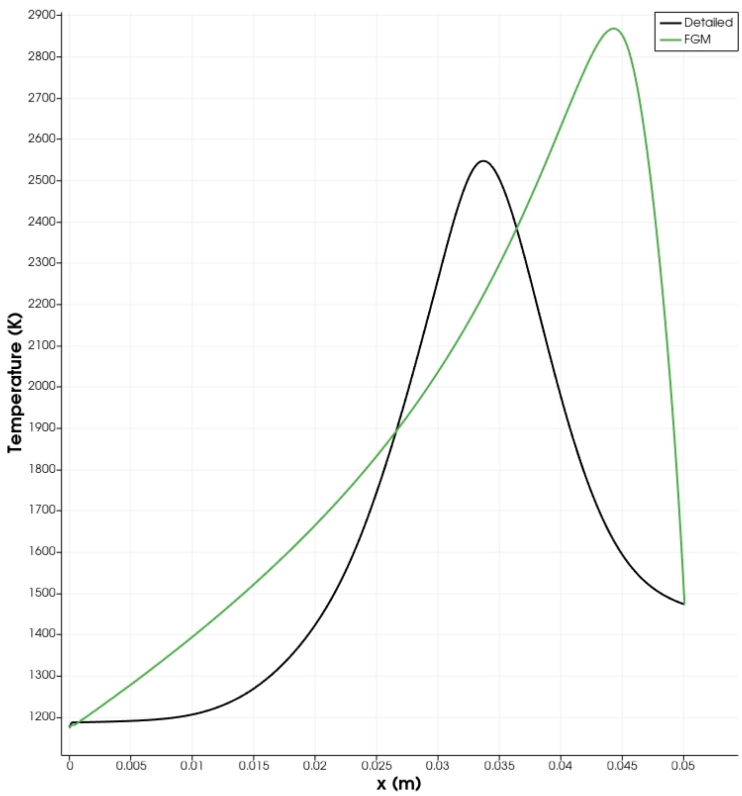


Figure 4.13: FGM simulation at 2 sec

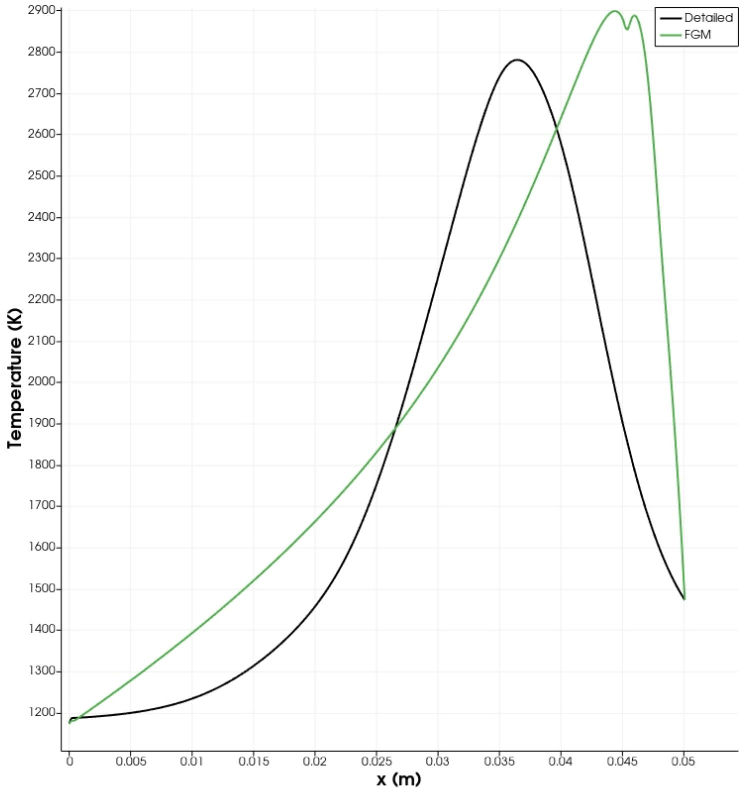


(a) 0.1 seconds

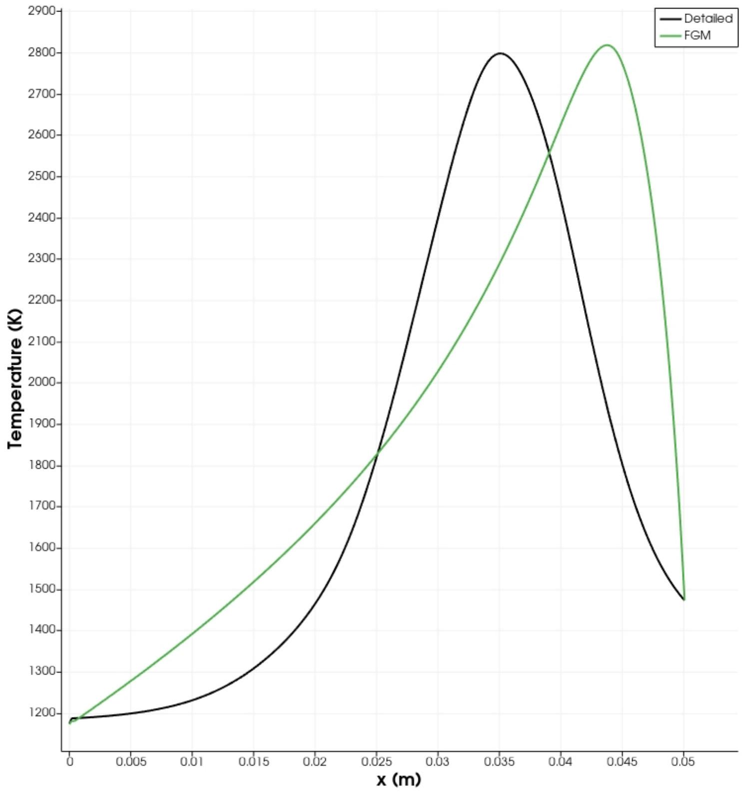


(b) 0.5 seconds

Figure 4.14: Temperature plot comparison at y=0 and z=0 cm for detailed chemistry and FGM simulation.

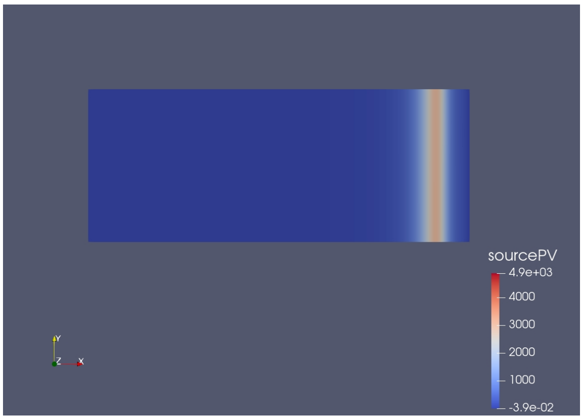


(a) 1 second

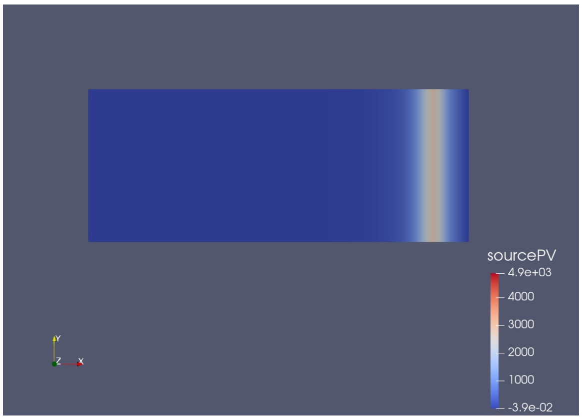


(b) 2 seconds

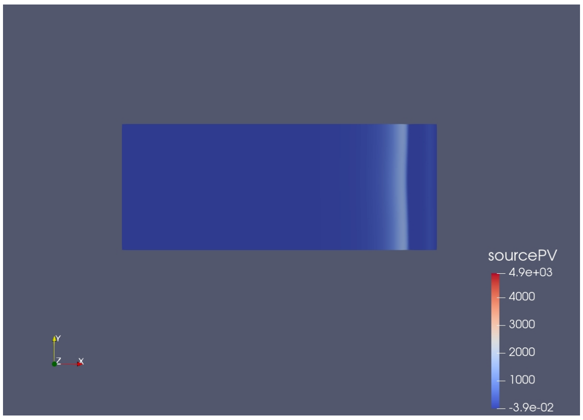
Figure 4.15: Temperature plot comparison at y=0 and z=0 cm for detailed chemistry and FGM simulation.



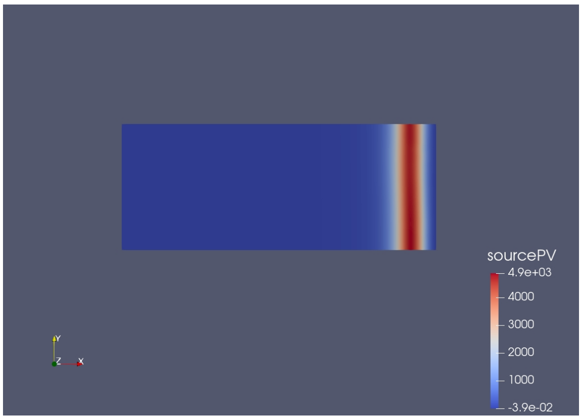
(a) 0.1 seconds



(b) 0.5 seconds



(c) 1 second



(d) 2 seconds

Figure 4.16: Source PV terms at z=0 and different simulation times

Chapter 5

Conclusion and Remarks

- The combustion of pulverised coal particles was realised as the combustion of volatile matter released from the coal during.after devolatisation.
- Non-premixed flamelets were made for the fuel composition and as a result a reliable FGM table is generated with the fuel taking the conditions of the properties of the volatile matter.
- The FGM table when validated against detailed chemistry solution in chem1d produced good results.
- A rudimentary adaptation of the FGMFoam is realised along with the FanzyFGM bi-linear interpolation lookup on OpenFOAM 2.4.x.
- A simplified geometry is used to compare detailed chemistry simulations with FGM on OpenFOAM.
- The FGM model on OpenFOAM needs to be investigated further in detail to confirm proper coupling with the FGM database. A positive side of this is that the table is looked up and data is retrieved (some) data from the table (Z, PV and T).
- Looking at coal combustion from computational cost perspective, FGM saves more than 5x time for a small simulation as this study, the accuracy should be definitely improved but even at the current stage, it is very promising.

Further Outlook

Since this study was done for a very simplified geometry. The following pathway would be in future be more promising :

- Investigation of the cause of the improper coupling of FGM in OpenFOAM.
- Adding a turbulence model and expanding it to the test geometry being used with the project 1.2

- Adding the effect of coal gasification by introducing a second stream of fuel (see figure 5.1), this contains the products of the char gasification reaction(CO + N2). This will complete the gaseous combustion stage of coal. A new transported mixture fraction($Z_{net} = Z_1 + Z_2$), describes the total mixture fraction of the gaseous mixture and a scalar(A) is introduced varying between 0 and 1. The value of A will give the fuel composition as defined as in the equation 5.1 (A = and also a controlling variable for an FGM table instead of mixture fraction D.

$$Y_{net} = Y_1(1 - A) + Y_2(A), \qquad A = [0 - 1], \tag{5.1}$$

here, Y_1 is the fuel composition of stream 1, Y_2 is the fuel composition of stream 2, Y_{net} is the fuel composition for the mixture of both streams as calculated in the flow solver.

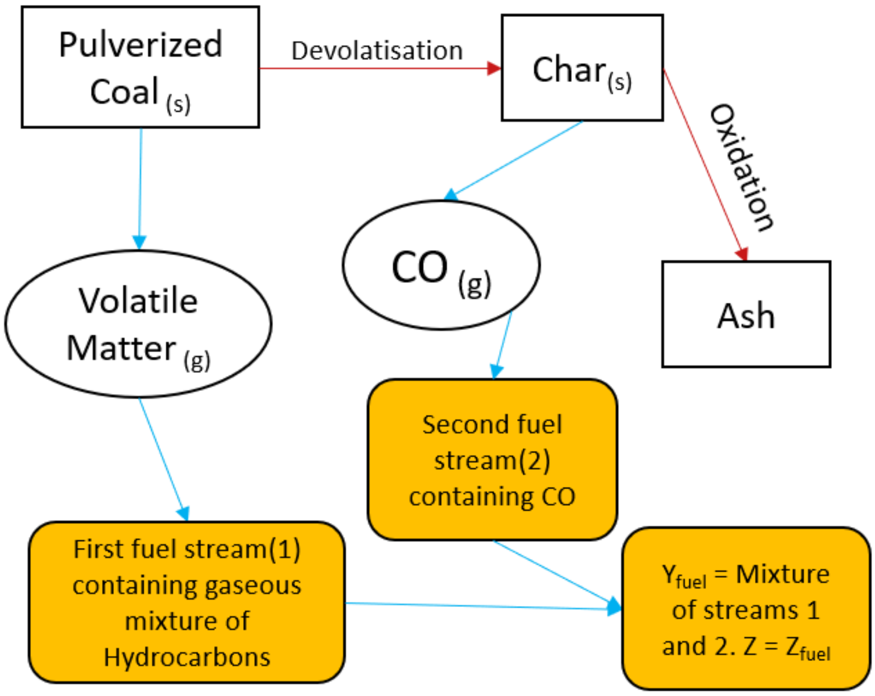


Figure 5.1: Coal gasification reaction scheme

- Increasing the dimension of the FGM table to include the effect of char gasification by implementing the method used by J. Watanabe and K. Yamamoto [25]

- Migrate the FGMFoam solver to newer versions of OpenFOAM to make use of easily available support than the older versions.

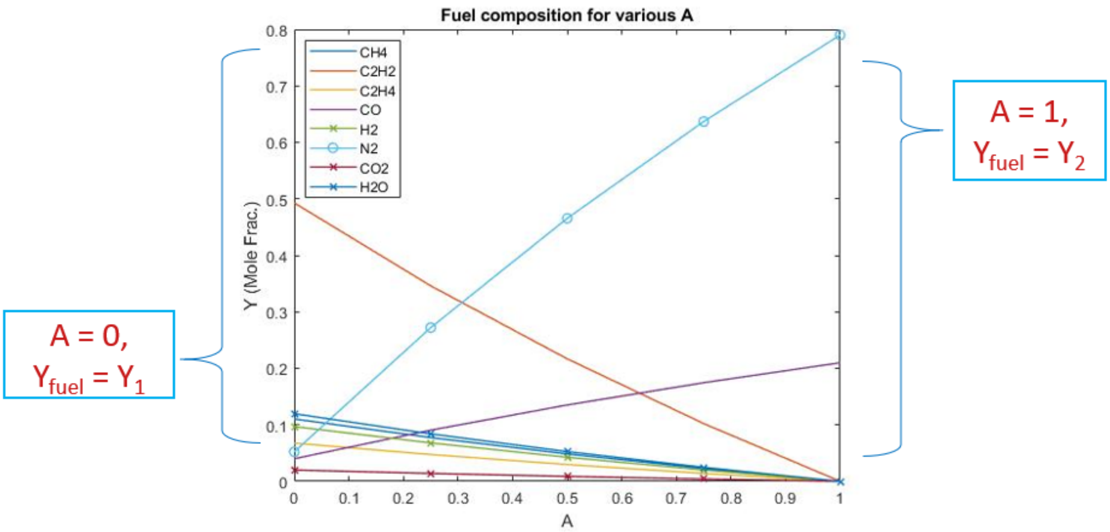


Figure 5.2: Fuel composition of mixture of two streams

References

(1) Eurofer - Steel Statistics, https://www.eurofer.eu/statistics/production-of-crude-steel/all-qualities/.

(2) Geerdes, M.; Chaigneau, R.; Kurunov, I., *Modern blast furnace ironmaking: an introduction (2015)*; Ios Press: 2015.

(3) Rangarajan, D.; Shiozawa, T.; Shen, Y.; Curtis, J. S.; Yu, A. *Industrial & Engineering Chemistry Research* **2014**, *53*, 4983–4990.

(4) Sarkar, S.; Gupta, G. S.; Kitamura, S.-y. *ISIJ international* **2007**, *47*, 1738–1744.

(5) GS, G.; Rudolph, V. *ISIJ international* **2006**, *46*, 195–201.

(6) Cui, J.; Hou, Q.; Shen, Y. *Powder Technology* **2020**, *362*, 539–549.

(7) Mondal, S.; Som, S.; Dash, S. *Journal of Physics D: Applied Physics* **2005**, *38*, 1301.

(8) Mathieson, J. G.; Truelove, J. S.; Rogers, H. *Fuel* **2005**, *84*, 1229–1237.

(9) Fu, D.; Chen, Y.; Zhou, C. Q. *Applied Mathematical Modelling* **2015**, *39*, 7554–7567.

(10) Yu, X.; Shen, Y. *Chemical Engineering Science* **2019**, *199*, 50–63.

(11) Fernando, R. *CCC/232 ISBN* **2014**, 978–92.

(12) Zhang, W. *Processes* **2016**, *4*, 36.

(13) Yoshitani, Y. *IFAC Proceedings Volumes* **1988**, *21*, 103–109.

(14) Lu, C.-Y.; Du, S.; Kuo, S.-K. *China Steel Technical Report* **2010**, *23*, 25–30.

(15) Rüther, H.-P.; Sancho, L.; Perez, R.; CHIAROTTI, U.; PERINI, P.; MIELENZ, O.; KÖCHNER, H. *EUR* **2007**.

(16) Jimenez, J.; Mochon, J.; de Ayala, J. S. *ISIJ international* **2004**, *44*, 518–526.

(17) Zhou, C.; Tang, G.; Wang, J.; Fu, D.; Okosun, T.; Silaen, A.; Wu, B. *JOM* **2016**, *68*, 1353–1362.

(18) Zhang, S.; Yu, A.; Zulli, P.; Wright, B.; Austin, P. *Applied Mathematical Modelling* **2002**, *26*, 141–154.

(19) Yang, K.; Choi, S.; Chung, J.; Yagi, J.-i. *ISIJ international* **2010**, *50*, 972–980.

(20) Dong, X.; Yu, A.; Chew, S. J.; Zulli, P. *Metallurgical and materials transactions B* **2010**, *41*, 330–349.

(21) Sankar, G.; Kumar, D. S.; Balasubramanian, K. *Fuel* **2019**, *236*, 643–665.

(22) Nasato, D. S.; Goniva, C.; Pirker, S.; Kloss, C. *Procedia Engineering* **2015**, *102*, 1484–1490.

(23) Van Oijen, J.; De Goey, L. *Combustion Science and Technology* **2000**, *161*, 113–137.

(24) Ubhayakar, S. K.; Stickler, D. B.; Von Rosenberg Jr, C. W.; Gannon, R. E. In *Symposium (international) on combustion*, 1977; Vol. 16, pp 427–436.

(25) Watanabe, J.; Yamamoto, K. *Proceedings of the Combustion Institute* **2015**, *35*, 2315–2322.

(26) McAllister, S.; Chen, J.-Y.; Fernandez-Pello, A. C., *Fundamentals of combustion processes*; Springer: 2011; Vol. 302.

(27) Ma, L. Computational modeling of turbulent spray combustion, https://repository.tudelft.nl/islandora/object/uuid:c1c27066-a205-45f4-a7b4-e36016bc313a/.

(28) Somers, L. The simulation of flat flames with detailed and reduced chemical models, English, Ph.D. Thesis, Department of Mechanical Engineering, 1994.

(29) Van Oijen, J. Chem1D-Manual, Dept. of Mechanical Engineering, TU Eindhoven, (Power and Flow Gitlab repository).

(30) De Goey, L.; ten Thije Boonkkamp, J. *Combustion and flame* **1999**, *119*, 253–271.

(31) Taylor, R.; Krishna, R., *Multicomponent mass transfer*; John Wiley & Sons: 1993; Vol. 2.

(32) Wilke, C. *The journal of chemical physics* **1950**, *18*, 517–519.

(33) Warnatz, J.; Maas, U.; Dibble, R. Combustion Springer-Verlag, 1996.

(34) Chemkin-III, R. *Reaction Design* **1999**.

(35) GRI-Mech 3.0, http://combustion.berkeley.edu/gri-mech/version30/text30.html#cite, (Accessed on 01/14/2021).

(36) Vascellari, M.; Tufano, G.; Stein, O.; Kronenburg, A.; Kempf, A.; Scholtissek, A.; Hasse, C. *Fuel* **2017**, *201*, 29–38.

(37) Vascellari, M.; Xu, H.; Hasse, C. *Proceedings of the Combustion Institute* **2013**, *34*, 2445–2452.

(38) Rieth, M.; Kempf, A. M.; Stein, O. T.; Kronenburg, A.; Hasse, C.; Vascellari, M. *Proceedings of the Combustion Institute* **2019**, *37*, 2927–2934.

(39) Donini, A. Advanced turbulent combustion modeling for gas turbine application, English, Ph.D. Thesis, Department of Mechanical Engineering, 2014.

(40) Unofficial OpenFOAM wiki, www.openfoamwiki.net.

(41) Caretto, L.; Gosman, A.; Patankar, S.; Spalding, D. In *Proceedings of the third international conference on numerical methods in fluid mechanics*, 1973, pp 60–68.

(42) Issa, R. I. *Journal of computational physics* **1986**, *62*, 40–65.

(43) Fancello, A.; Bastiaans, R.; de Goey, L. In *AIP Conference Proceedings*, 2013; Vol. 1558, pp 168–171.

(44) Fancello, A. **2014**.

(45) Fancello, A.; Panek, L.; Lammel, O.; Krebs, W.; Bastiaans, R. J.; de Goey, L. In *Turbo Expo: Power for Land, Sea, and Air*, 2014; Vol. 45691, V04BT04A012.

(46) Hwang, S. m.; Kurose, R.; Akamatsu, F.; Tsuji, H.; Makino, H.; Katsuki, M. *Energy & Fuels* **2005**, *19*, 382–392.

(47) Rieth, M.; Clements, A.; Rabaçal, M.; Proch, F.; Stein, O.; Kempf, A. *Proceedings of the Combustion Institute* **2017**, *36*, 2181–2189.

(48) Bilnear interpolation, https://en.wikipedia.org/wiki/Bilinear$_{i}nterpolation$.

Appendix A

Appendix - Flamelets Generation

Matlab Code - 1

```
%% Set the variable range
clear all; clc ; close all;

valstrt = 0.1; % Start value strainrate
valend  = 200000; % End value strainrate
absd    = 1; % Absolute change
reld    = 5e-2; % Relative change

linear=0;

if linear
    val = (valstrt:absd:valend);
else
    N = ceil(abs(log(valend/valstrt))/reld);
    val = valstrt*exp(reld*(0:N));
    val(end) = valend;
end

N = length(val);

plot(val,'.-')

% Some other settings
outdir = './test';
startchem1dcmd = 'chem1d';

dlmwrite('valsseries.txt',val,'delimiter','\n');
```

```
%-----------------------------------------------------------%
```

The above code is used to generate a file with values of strain rate which was passed to a shell script which would call chem1d for all these values of strain rate. The script below changes the name of the output solution files and names them in an chronological order.

Shell Script - 1

```bash
#!/bin/bash

OUTPUT_DIR='Output'
mkdir -p Output

while IFS="" read -r i || [ -n "$i" ]
do
    awk '{sub(/VARIABLE_NAME/,'$i')}1' settings_template.csf > settings.csf
    chem1d
    cp siend.dat Output/siend-$(printf %08.2f $i | sed s/[.]//g).dat
    cp yiend.dat Output/yiend-$(printf %08.2f $i | sed s/[.]//g).dat
  # cp lambdacp.dat Output/lambdacp-$(printf %08.2f $i | sed s/[.]//g).dat
  # cp chem1d.log Output/chem1d-$(printf %08.2f $i | sed s/[.]/_/g).log
    printf 'chem1d %08.2e\n' "$i"
done < $1
```

Appendix B

Appendix - Chem1D Settings

```
[ADDITIONAL_USERCOMMENT]
counterflow test flame for FGM gen

[GRID_LEFTBOUNDARY]                     => Typical value: -0.5 cm
-6
[GRID_RIGHTBOUNDARY]                    => Typical value : 2.0 cm
6

[GRID_NUMBEROFPOINTS]                   => Typical value: 100
300

[MODEL_FLAMETYPE]                       => Options: FREE, COUNTERFLOW .ETC
COUNTERFLOW

[MODEL_CHEMISTRY]                       => Options: DETAILED, FGM
DETAILED

[MODEL_TRANSPORT]                       => Options: LEWIS, MIXTUREAVERAGED,
    COMPLEX
LEWIS

[MODEL_SIMULATIONTYPE]                  => Options: TIMEDEPENDENT,
    QUASITIMEDEPENDENT, STATIONARY
STATIONARY

[GASMIXTURE_FUELCOMPOSITION]            => Unit : MASS FRACTIONS
CH4     0.082
C2H2    0.595
C2H4    0.088
```

```
CO              0.052
H2              0.009
N2              0.034
CO2             0.041
H2O             0.099
[ENDOF_FUELCOMPOSITION]

[GASMIXTURE_OXIDIZERCOMPOSITION]        => Unit : MASS fractions
O2      0.21
N2      0.79
[ENDOF_OXIDIZERCOMPOSITION]

[BOUNDARY_INLETPRESSURE]                => pressure [N/m2]
1.013250E5

[BOUNDARY_INLETTEMPERATURE]             => FUEL AND OXIDISER TEMPERATURES [K]
1173 1473

[STRAINANDCURVATURE_STRAIN]             => Unit : 1/s
10000

[PREPROCESSING_STARTSOLUTIONFILE]       => Typical value: yistart.dat
yiend.dat

[POSTPROCESSING_OUTPUTFILES]            => Supply a filename to the default ouput
     files
MASS     yiend.dat     ON
MOLE     xiend.dat     OFF
CONC     ciend.dat     OFF
SOURCE   siend.dat     ON
LEWIS    lewis.dat     OFF
EQUIL    yiequil.dat   OFF
LAMBDACP lambdacp.dat  ON
REACTIONRATES rates.dat OFF
RESIDUAL residuals.dat OFF
[ENDOF_OUTPUTFILES]

[MODEL_CHEMISTRY]                       => Options: DETAILED, FGM
DETAILED

[MODEL_TRANSPORT]                       => Options: LEWIS, MIXTUREAVERAGED,
```

```
    COMPLEX
LEWIS

[MODEL_SIMULATIONTYPE]                      => Options: TIMEDEPENDENT,
    QUASITIMEDEPENDENT, STATIONARY
TIMEDEPENDENT

#
# Timestep Control
#
[TIMESTEPPER_INITIALTIMESTEP]               => Typical value 1.0e-6
1.0E-06
[TIMESTEPPER_MINIMUMTIMESTEP]               => Typical value 1.0e-10
1.0E-16
[TIMESTEPPER_MAXIMUMTIMESTEP]               => Typical value 1.0e-4
1.0E-04
[TIMESTEPPER_MAXSIMULATIONTIME]             => Typical value 1.0
1.0E+00
[TIMESTEPPER_OUTPUTSTEP]                    => Typical value 100
20
```

Appendix C

Appendix - OpenFOAM Conditions

0/Z

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.4.x                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.org                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       volScalarField;
    location    0;
    object      Z;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

dimensions      [0 0 0 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    fuel
    {
        type            fixedValue;
        value           uniform 1.0;
```

```
    }
    air
    {
        type            fixedValue;
        value           uniform 0.0;
    }
    outlet
    {
        type            inletOutlet;
            value                   uniform 0;
            inletValue      uniform 0;
    }
    frontAndBack
    {
        type            empty;
    }
}


// ************************************************************************* //
```

0/PV

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.4.x                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.org                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       volScalarField;
    location    0;
    object      PV;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

dimensions      [0 0 0 0 0 0 0];

internalField   uniform 0;

boundaryField
{
    fuel
    {
        type            fixedValue;
        value           uniform 0;
    }
    air
    {
        type            fixedValue;
        value           uniform 0;
    }
    outlet
    {
        type            inletOutlet;
        value           uniform 0;
        inletValue      uniform 0;
    }
```

```
    frontAndBack
    {
        type            empty;
    }
}


// ************************************************************************* //
```

0/U

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.4.0                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.org                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       volVectorField;
    location    0;
    object      U;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

dimensions      [0 1 -1 0 0 0 0];

internalField   uniform (0 0 0);

boundaryField
{
    fuel
    {
        type            fixedValue;
        value           uniform (0.1 0 0);
    }
    air
    {
        type            fixedValue;
        value           uniform (-0.1 0 0);
    }
    outlet
    {
        type            pressureInletOutletVelocity;
        value           uniform (0 0 0);
    }
    frontAndBack
```

```
    {
        type            empty;
    }
}


// ************************************************************************* //
```

0/T

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.4.0                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.org                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       volScalarField;
    location    0;
    object      T;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

dimensions      [0 0 0 1 0 0 0];

internalField   uniform 1173;

boundaryField
{
    fuel
    {
        type            fixedValue;
        value           uniform 1173;
    }
    air
    {
        type            fixedValue;
        value           uniform 1473;
    }
    outlet
    {
        type            inletOutlet;
        inletValue      uniform 1173;
        value           uniform 1173;
    }
```

```
    frontAndBack
    {
        type            empty;
    }
}


// ************************************************************************* //
```

system/fvSchemes

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.4.0                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.org                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    location    system;
    object      fvSchemes;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

ddtSchemes
{
    default         Euler;
}

gradSchemes
{
    default         Gauss linear;
}

divSchemes
{
    default         none;

    div(phi,U)      Gauss limitedLinearV 0.5;
    div(phi,Yi_h)   Gauss limitedLinear 1;
    div(phi,K)      Gauss limitedLinear 1;
    div(phid,p)     Gauss limitedLinear 0.3;
    div(phi,epsilon) Gauss limitedLinear 1;
    div(phi,k) Gauss limitedLinear 1;
    div((muEff*dev2(T(grad(U))))) Gauss linear;
    div(phi,PV)     Gauss limitedLinear01 0.5;
```

```
    div(phi,Z)     Gauss limitedLinear01 0.5;
    div(phi,varPV)  Gauss limitedLinear01 0.5;
    div(((rho*nuEff)*dev2(T(grad(U))))) Gauss linear;
}

laplacianSchemes
{
    default         Gauss linear orthogonal;
}

interpolationSchemes
{
    default         linear;
}

snGradSchemes
{
    default         orthogonal;
}

fluxRequired
{
    default         no;
    p;
}


// ************************************************************************* //
```

system/fvSolution

```
/*---------------------------------*- C++ -*-----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.4.0                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.org                      |
|    \\/     M anipulation  |                                                 |
\*-----------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    location    system;
    object      fvSolution;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

solvers
{
    rho
    {
        solver          PCG;
        preconditioner  DIC;
        tolerance       1e-06;
        relTol          0.1;
    }

    rhoFinal
    {
        $rho;
        tolerance       1e-06;
        relTol          0;
    }

    p
    {
        solver           PCG;
        preconditioner   DIC;
        tolerance        1e-6;
```

```
        relTol          0.001;
    }

    pFinal
    {
        $p;
        tolerance       1e-6;
        relTol          0.0;
    }

    (U|h|hs|varPV|PV|Z|YCO|k|epsilon)
    {
        solver          PBiCG;
        preconditioner  DILU;
        tolerance       1e-06;
        relTol          0.001;
    }

    (U|h|hs|varPV|PV|Z|YCO|k|epsilon)Final
    {
        solver          PBiCG;
        preconditioner  DILU;
        tolerance       1e-06;
        relTol          0;
    }

/*
    (U|h|hs|PV|Z|k|epsilon)
    {
        solver          smoothSolver; //GAMG;
//        preconditioner  FDIC;
        smoother        DILU;
//        agglomerator    faceAreaPair;
//        nCellsInCoarsestLevel 100;
        tolerance       1e-6;
        relTol          0.01;
//        mergeLevels     1;
    }


    (U|h|hs|PV|Z|k|epsilon)Final
```

```
    {
        $(U|h|hs|PV|Z|k|epsilon);
        relTol          0.01;
    }

*/
    Yi
    {
        $hsFinal;
    }
}

PIMPLE
{
    momentumPredictor yes;
    nOuterCorrectors  2;
    nCorrectors     3;
    nNonOrthogonalCorrectors 0;
}

// ************************************************************************* //
```

constant/fgmProperties

```
/*--------------------------------*- C++ -*----------------------------------*\
| =========                 |                                                 |
| \\      /  F ield         | OpenFOAM: The Open Source CFD Toolbox           |
|  \\    /   O peration     | Version:  2.0.1                                 |
|   \\  /    A nd           | Web:      www.OpenFOAM.com                      |
|    \\/     M anipulation  |                                                 |
\*---------------------------------------------------------------------------*/
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    location    constant;
    object      fgmProperties;
}
// * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * * //

debugOutput             on;

fgmFilePath             ./constant/database.fgm;

fgmDimension            2;

nVar                    17;

Sct                     Sct [0 0 0 0 0 0 0] 1;

inflow                  INLET;

outflow                 OUTLET;


fgmThermoTransportIndices
(
    10 // Density
    16 // Compressibility                   | psi=1/(RT)
    11 // Temperature                       | T
    12 // Heat capacity at constant pressure | Cp
    13 // Thermal Diffusivity               | \alpha
```

```
    14 // Viscosity                          | \mu
    5  // Differential diffusion coefficient for PV | diffPV
    3  // SourcePV
    6 // CH4
    9 // O2
    8 // CO
    7 // C2H2
);


// ********************************************************************** //
```

Appendix D

Appendix - Misc.

This code gives the final composition of the fuel in case of two mixture fractions, useful for next stage when the scalar 'A' is incorporated in the FGM table. This fuel composition will be useful to make the FGM table for the two-fuel stream case.

```
%Proximate analysis
clc; clear all; close all;

c.sample = 100; %grams

c.ASH = 0.152;
c.H2O = 0.026;
c.VM = 0.269;
c.C = 0.579;

m.vol = c.VM*c.sample;
m.h2o = c.H2O*c.sample;
m.char = c.sample - m.vol - m.h2o;

alpha = 3.76;   %for air .21 O2 and .79 N2
                % see Watanabe Flamelet

%% Init Y_vol

%Enter the (mass) fractions of the vol matter

Y.name = [CH4, C2H2, 'C2H4, CO, H2, N2, CO2, H2O];

Y.vol = [0.082 0.595 0.088 0.052 0.009 0.034 0.041 0.1];
Y.char = [0 0 0 .347 0 0.653 0 0];

```

```
Y.vol = Y.vol / sum(Y.vol);
Y.char = Y.char / sum(Y.char);
Mwt.vol =[16 26 28 28 2 14 44 18];
%% NEW  A = 0 - 1;
A = [0 0.25 0.5 0.75 1];
Y_fuel = {};
Y.fuel = zeros(1,length(Y.vol));
for i = 1:length(A)
    for j = 1:length(Y.vol)
    Y.fuel(j) = (1 - A(i))*(Y.vol(j)) + A(i)*Y.char(j);
    end
Y_fuel = [Y_fuel Y.fuel]
end
%% Calculation of A and Z_max
%
% beta = [0 0.25 0.5 1]; %amount of char gasified/ influence of char
% massbetachar = m.char*beta;
%
% A = zeros(size(beta));
%
% for i = 1:1:length(beta)
%
%     A(i) = ((m.char)*(beta(1,i)))/(m.vol);
%
% end
%
% Z_max = zeros(size(beta));
%
% for i = 1:1:length(beta)
%
%     Z_max(i) = (1 + A(i))/(1 + A(i) + ((32 + A(i)*alpha*28)/(12))*A(i) );
```

```
%
% end
%
% %% Fuel composition
%
% Y_fuel = {};
%
% for j = 1:length(A)
%
%     A_calc = A(j);        %Enter the A acc. to beta value to get the fuel comp.
%
%     Y.fuel = zeros( size(Y.vol) );
%
%     for i = 1:length(Y.fuel)
%
%         Y.fuel(i) = ( Y.vol(i) + (A_calc + ((32 + alpha*28)/24)*A_calc ...
%             )*Y.char(i)) / ( 1 + A_calc + ((32 + alpha*28)/24)*A_calc);
%
%     end
%
% Y_fuel = [Y_fuel Y.fuel]
%
% end
%
%% Conversion of MassFrac to MoleFrac

for i = 1:length(Y_fuel)

    Y_fuel{i} = Y_fuel{i}() ./ Mwt.vol;

    Y_fuel{i} = Y_fuel{i} / sum(Y_fuel{i});

end


%% Plot diff. fuel comp ( % vs A )

Y_F = [];

for i = 1:length(A)

```

```
    temp = Y_fuel{i};
    Y_F = [Y_F; temp];

end


% for i = 1:length(Y.vol)
%
% plot(A,Y_F(:,i))
% hold on
%
% end

plot(A,Y_F(:,1),'--','LineWidth',1)
hold on
plot(A,Y_F(:,2),'--','LineWidth',1)
hold on
plot(A,Y_F(:,3),'--','LineWidth',1)
hold on
plot(A,Y_F(:,4),'--','LineWidth',1)
hold on
plot(A,Y_F(:,5),'x--','LineWidth',1)
hold on
plot(A,Y_F(:,6),'o--','LineWidth',1)
hold on
plot(A,Y_F(:,7),'x--','LineWidth',1)
hold on
plot(A,Y_F(:,8),'x--','LineWidth',1)
hold on

title('Fuel composition for various A')
xlabel('A')
ylabel('Y')
legend('CH4', 'C2H2', 'C2H4', 'CO', 'H2', 'N2', 'CO2', 'H2O')

%% Write fuel comp. to text file

fileID = fopen('Y_fuel.txt','w');

fprintf(fileID,'Fuel composition for varying A \n \n');

```

```
for j = 1:length(Y_fuel)

    fprintf(fileID,'%-4s %-5.6f \n \n', A = , A(j));

    for i = 1:length(Y_fuel{1})

        fprintf(fileID,'%-6s \t %5.4f\n', Y.name(i), Y_fuel{j}(i));

    end

    fprintf(fileID, '\n \n');
end


%--------------------------------------------------------------------------%
```

PIMPLE Algorithm

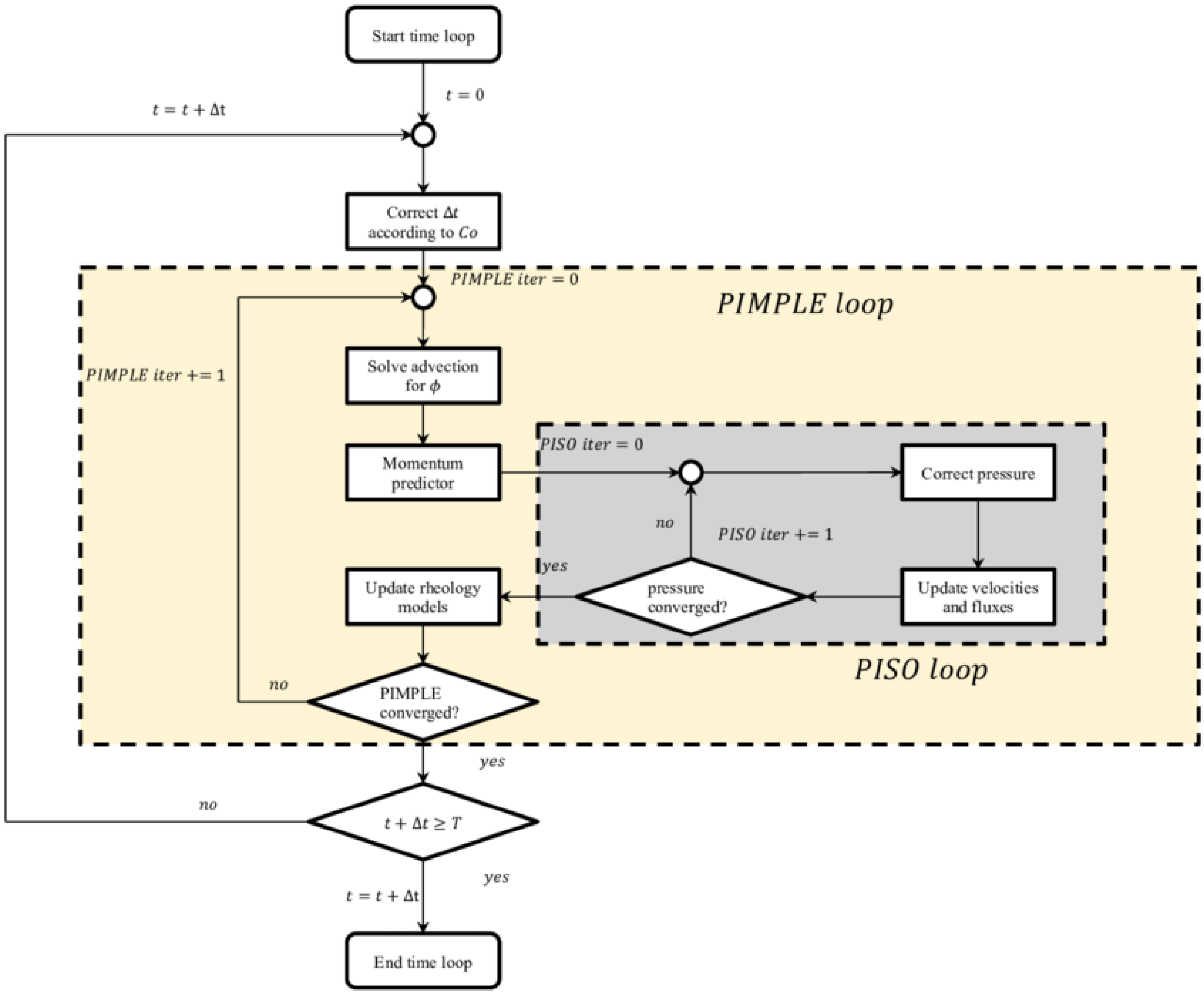


Figure D.1: Simplified PIMPLE algorithm flowchart

Bilinear Interpolation

The following equations show the algorithm of bilinear interpolation used in the FanzyFGM function used in OpenFOAM. In the figure D.2 the red dots show the data points in a rectilinear grid and the green point is the one to be interpolated. Bilinear interpolation is an extension of linear interpolation algorithm for a rectilinear 2D grid. In this algorithm, linear interpolation is performed in one direction and then again in the other direction.[48]

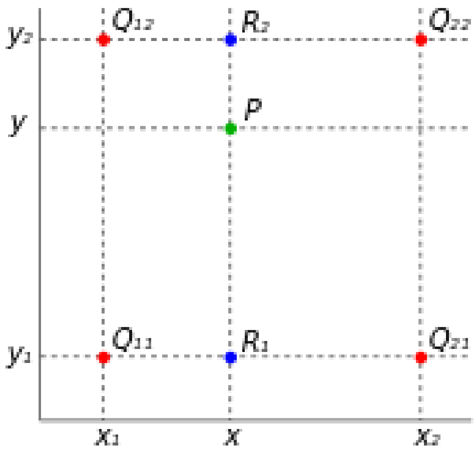


Figure D.2: Bilinear Interpolation - schematic

$$
\begin{aligned}
f(x, y_1) &\approx \frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}), \\
f(x, y_2) &\approx \frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}).
\end{aligned}
$$

Linear interpolation is first done in x-direction. Here, $Q_{11} = (x_1, y_1)$, $Q_{12} = (x_1, y_2)$, $Q_{21} = (x_2, y_1)$ and $Q_{22} = (x_2, y_2)$

$$
\begin{aligned}
f(x, y) &\approx \frac{y_2 - y}{y_2 - y_1} f(x, y_1) + \frac{y - y_1}{y_2 - y_1} f(x, y_2) \\
&= \frac{y_2 - y}{y_2 - y_1} \left(\frac{x_2 - x}{x_2 - x_1} f(Q_{11}) + \frac{x - x_1}{x_2 - x_1} f(Q_{21}) \right) + \frac{y - y_1}{y_2 - y_1} \left(\frac{x_2 - x}{x_2 - x_1} f(Q_{12}) + \frac{x - x_1}{x_2 - x_1} f(Q_{22}) \right) \\
&= \frac{1}{(x_2 - x_1)(y_2 - y_1)} \left(f(Q_{11})(x_2 - x)(y_2 - y) + f(Q_{21})(x - x_1)(y_2 - y) + f(Q_{12})(x_2 - x)(y - y_1) + f(Q_{22})(x - x_1)(y - y_1) \right) \\
&= \frac{1}{(x_2 - x_1)(y_2 - y_1)} \begin{bmatrix} x_2 - x & x - x_1 \end{bmatrix} \begin{bmatrix} f(Q_{11}) & f(Q_{12}) \\ f(Q_{21}) & f(Q_{22}) \end{bmatrix} \begin{bmatrix} y_2 - y \\ y - y_1 \end{bmatrix}.
\end{aligned}
$$